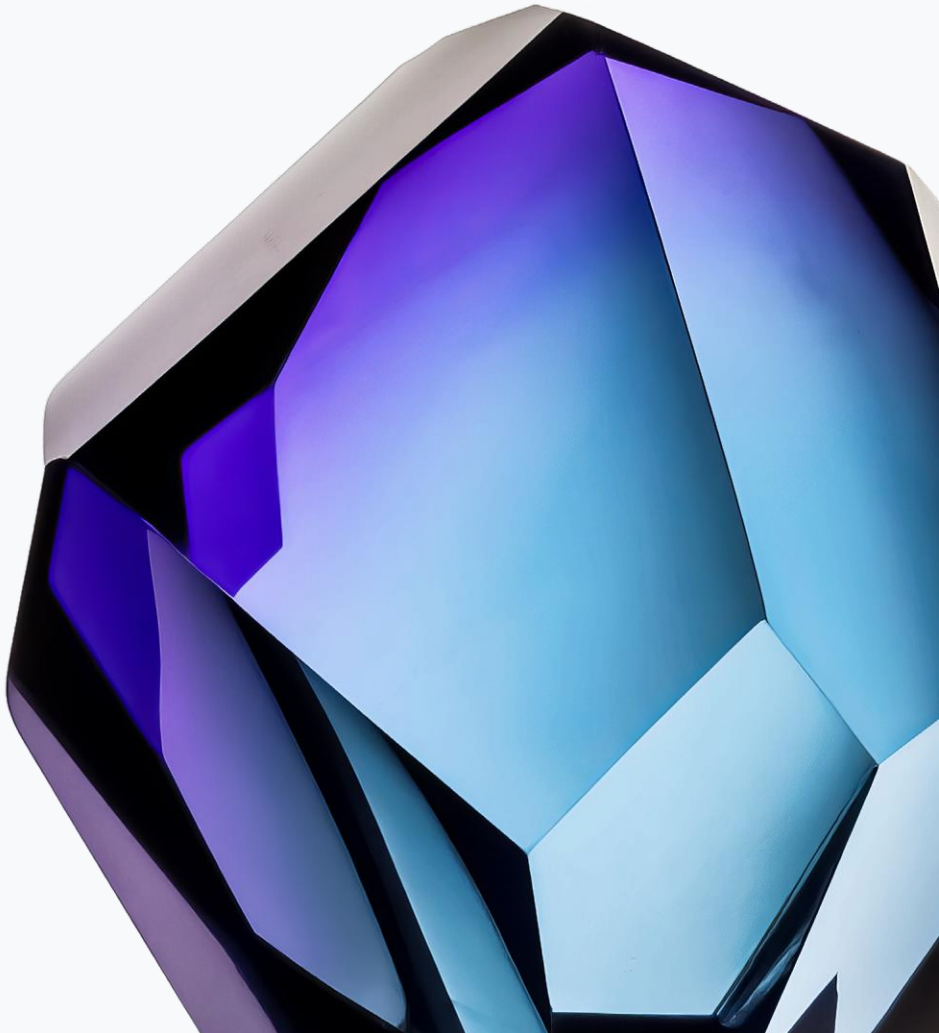
Промпт инжиниринг



О чем это руководство



Промтинг (prompt engineering) — это больше, чем техника. Это язык общения с разумом машин. Это мост между человеческим замыслом и цифровым исполнением. Это ключ, открывающий двери к полноценной автоматизации, созданию ИИ-агентов и построению сложных цепочек решений без необходимости писать ни строчки кода. Именно он превращает нейросеть из загадочной "чёрной коробки" в послушного и продуктивного союзника.

Как в программировании важен синтаксис, так в работе с ИИ важен промт. Правильно сформулированный запрос — это алгоритм, стратегия, бизнес-процесс в миниатюре. Это интерфейс между вашим мышлением и возможностями нейросети.

В этом руководстве вы узнаете, почему **промтинжиниринг — это главный навык нового времени**, как он влияет на эффективность работы с нейросетями, автоматизацию, создание ИИ-агентов, и почему будущее принадлежит тем, кто умеет разговаривать с машинами на их языке — но по-человечески понятно.

Вам не нужно быть ученым или инженером машинного обучения — каждый может написать промпт

Введение

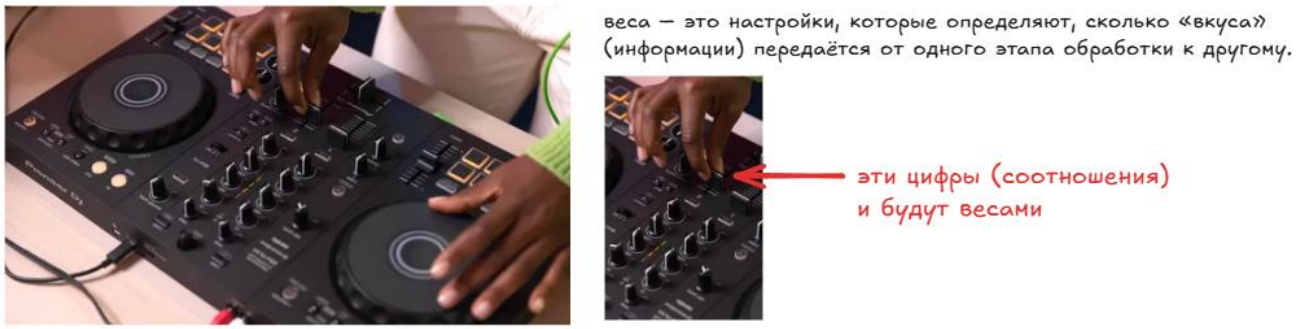
Размышляя о вводе и выводе большой языковой модели, текстовый промпт (иногда сопровождаемый другими модальностями, такими как изображения) является входом, используемым моделью для предсказания конкретного вывода. Вам не нужно быть ученым по данным или инженером машинного обучения — каждый может написать промпт. Однако создание наиболее эффективного промпта может быть сложным. Множество аспектов вашего промпта влияют на его эффективность: используемая модель, обучающие данные модели, конфигурации модели, выбор слов, стиль и тон, структура и контекст — все это имеет значение. Поэтому инженерия промптов — это итеративный процесс. Неадекватные промпты могут привести к неоднозначным, неточным ответам и могут помешать способности модели предоставлять значимый результат.

Когда вы общаетесь с нейросетью, вы, по сути, пишете промпты, это руководство фокусируется на написании промптов для любых моделей в режиме чата или с использованием API, поскольку при прямом взаимодействии с моделью у вас будет доступ к настройкам, таким как температура и т.д.

В этом руководстве подробно обсуждается инженерия промптов. Мы рассмотрим различные методы промптинга, которые помогут вам начать работу, и поделимся советами и лучшими практиками, чтобы стать экспертом в промптинге. Мы также обсудим некоторые проблемы, с которыми вы можете столкнуться при создании промптов.

Промпт-инжиниринг

Помните, как работает LLM? Это диджейский пульт, который с помощью тумблеров предсказывает следующее слово!



Модель принимает последовательный текст в качестве входных данных, а затем предсказывает, каким должен быть следующий токен на основе данных, на которых она была обучена. LLM заточена для повторения этого процесса, добавляя ранее предсказанный токен в конец последовательного текста для предсказания следующего токена. Предсказание следующего токена основано на отношении между тем, что находится в предыдущих токенах, и тем, что LLM видела во время своего обучения.

Когда вы пишете промпт, вы пытаетесь настроить LLM для предсказания правильной последовательности токенов. Промтинжиниринг — это процесс проектирования высококачественных промптов, которые направляют LLM для генерации точных выводов. Этот процесс включает в себя настройку для нахождения лучшего промпта, оптимизацию длины промпта и оценку стиля написания и структуры промпта по отношению к задаче. В контексте обработки естественного языка и LLM, промпт — это вход, предоставляемый модели для генерации ответа или предсказания.

Эти промпты могут использоваться для выполнения различных задач понимания и генерации, таких как обобщение текста, извлечение информации, вопросы и ответы, классификация текста, перевод языка или кода, генерация кода и документация кода или рассуждения.

При инженерии промптов вы начнете с выбора модели. Промпты могут нуждаться в оптимизации для вашей конкретной модели, независимо от того, используете ли вы языковые модели GPT, Claude, Gemini в Vefiex AI, Grok, Mistral или модель с открытым исходным кодом, такую как Gemma, DeepSeek, Qwen или LLaMA.

Кроме промпта, вам также потребуется настроить различные конфигурации LLM.

Конфигурация вывода LLM

После выбора модели вам нужно будет определить конфигурацию модели. Большинство LLM поставляются с различными опциями конфигурации, которые контролируют вывод LLM. Эффективная инженерия промптов требует оптимальной настройки этих конфигураций для вашей задачи.

Длина вывода

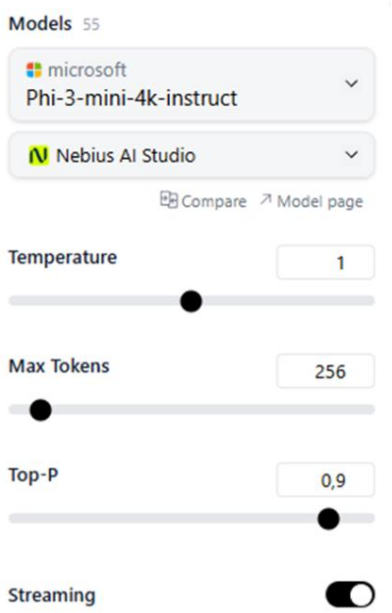
Важной настройкой конфигурации является количество токенов для генерации в ответе. Генерация большего количества токенов требует больше вычислений от LLM, что приводит к более высокому потреблению энергии, потенциально более медленному времени отклика и более высоким затратам.

Уменьшение длины вывода LLM не заставляет LLM становиться более стилистически или текстуально кратким в создаваемом выводе, оно просто заставляет LLM прекратить предсказание большего количества токенов, как только достигнут предел. Если ваши потребности требуют короткой длины вывода, вам также, возможно, потребуется инженеризировать ваш промпт для соответствия.

Ограничение длины вывода особенно важно для некоторых методов промптинга LLM, таких как ReAct, где LLM будет продолжать генерировать бесполезные токены после нужного вам ответа.

Имейте в виду, генерация большего количества токенов требует больше вычислений от LLM, что приводит к более высокому потреблению энергии и потенциально более медленному времени отклика, что приводит к более высоким затратам. В уроках по ЛЛМ моделям мы показывали вам как при уменьшении длины вывода меняется результат. Всегда держите в

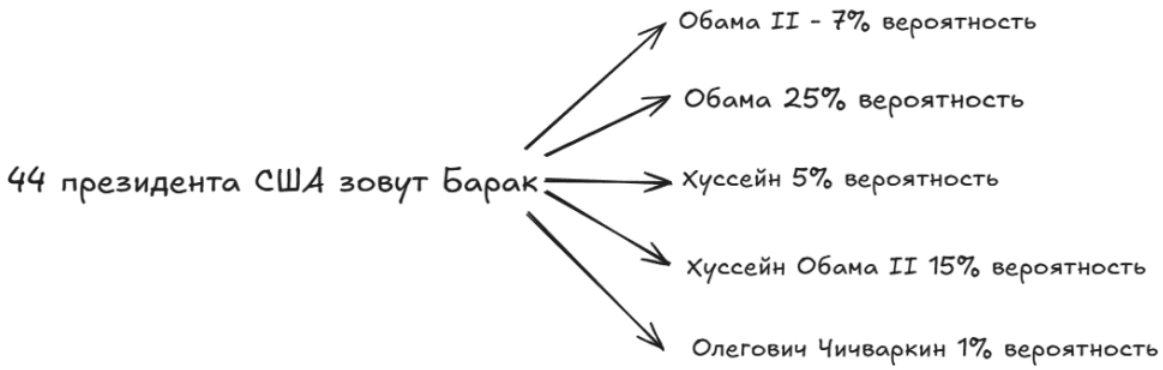
голове: чем меньше «max tokens», тем меньше модель «рассуждает» и при значении 5-10 может ошибиться в ответе.

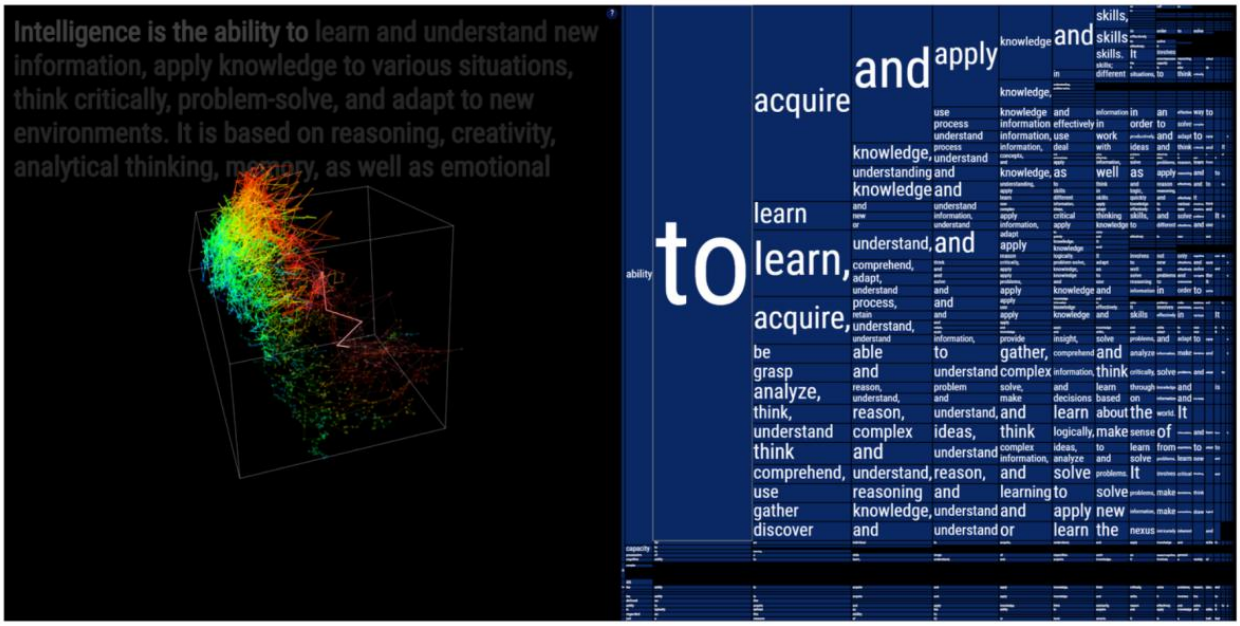


Управление выборкой

LLM формально не предсказывают один токен. Скорее, LLM предсказывают вероятности для того, каким может быть следующий токен, где каждый токен в словаре LLM получает вероятность. Затем из этих вероятностей токенов выбирается, чтобы определить, каким будет следующий произведенный токен.

Температура, top-K и top-P являются наиболее распространенными настройками конфигурации, которые определяют, как обрабатываются предсказанные вероятности токенов для выбора одного выходного токена.





Температура

Температура контролирует степень случайности в выборе токена. Более низкие температуры подходят для промптов, ожидающих более детерминированный ответ (страшное слово? Наиболее вероятный в переводе на человеческий, но далее будем использовать именно это понятие, так как слово вероятный не совсем уместно), в то время как более высокие температуры могут привести к более разнообразным или неожиданным результатам. Температура 0 (жадное декодирование): всегда выбирается токен с наибольшей вероятностью (хотя следует отметить, что если два токена имеют одинаковую наивысшую предсказанную вероятность, в зависимости от реализации правила разрешения ничьих, вы не всегда можете получить один и тот же вывод с температурой 0). На этом моменте у вас возник в глове вопрос – какая еще температура? Погодите, все по порядку, просто зафиксируйте этот момент.

Температуры, близкие к максимальным, имеют тенденцию создавать более случайный вывод. И по мере того, как температура становится все выше и выше, все токены становятся одинаково вероятными быть следующим предсказанным токеном.

Управление температурой у нейросети может быть понято аналогично функции softmax, используемой в машинном обучении (или функции «denoising» о которой мы поговорим в блоке по генерации изображений). Низкая настройка температуры отражает низкую температуру softmax (T), подчеркивая единственную, предпочтительную температуру с высокой уверенностью. Более высокая настройка температуры нейронки похожа на высокую температуру softmax, делая более широкий диапазон температур вокруг выбранной настройки более приемлемым. Эта повышенная неопределенность учитывает сценарии, где жесткая, точная температура может не быть необходимой, например, при экспериментах с творческими результатами.

Top-K and top-P

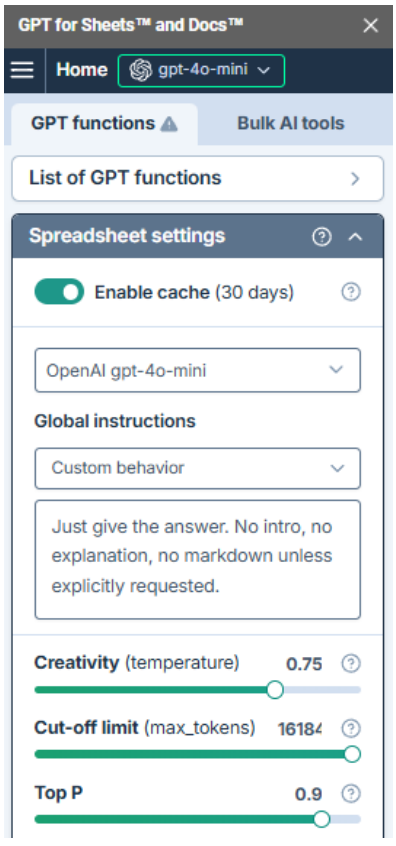
Top-K и top-P (также известные как выборка ядра) — это две настройки выборки, используемые в LLM для ограничения предсказанного следующего токена токенами с наивысшими предсказанными вероятностями. Как и температура, эти настройки выборки контролируют случайность и разнообразие генерируемого текста.

- **Top-K** выборка выбирает K наиболее вероятных токенов из предсказанного распределения модели. ***Чем выше top-K, тем более творческим и разнообразным будет вывод модели; чем ниже top-K, тем более ограниченным и фактическим будет вывод модели***. Top-K равный 1 эквивалентен жадному декодированию.

- **Top-P** выборка выбирает токены с наивысшей вероятностью, чья совокупная вероятность не превышает определенное значение (P). Значения для P варьируются от 0 (жадное декодирование) до 1 (все токены в «словаре» LLM).

Лучший способ выбора между top-K и top-P — экспериментировать с обоими методами (или обоими вместе) и посмотреть, какой из них дает результаты, которые вы ищете. Подробнее эти настройки мы разберем когда будем работать с АПИ моделей и в гугл таблицах.

И тут снова задаемся вопросом? О чем речь? Где в Chat GPT я что там выбираю? А теперь выдыхаем и вспоминаем шаблоны из гугл таблиц:



На самом деле вы уже работали с этими параметрами в таблицах, но я пока не давал вам как их можно регулировать. Просто откройте любой шаблон в GPT Sheets и вы увидите эти параметры. В режиме чата они тоже есть но скрыты для удобства и простоты интерфейса.

Объединяя все вместе

Выбор между top-K, top-P, температурой и количеством токенов для генерации зависит от конкретной задачи и желаемого результата, и все настройки влияют друг на друга. Также важно убедиться, что вы понимаете, как ваша выбранная модель комбинирует различные настройки выборки вместе. Как? Никто не знает, поверьте, даже разработчики в корпорациях. Просто эксперементируйте.

- Если температура, top-K и top-P все доступны, токены, которые соответствуют критериям top-K и top-P, являются кандидатами на следующий предсказанный токен, а затем температура («denoising» или «фантазия нейросети») применяется для выборки из токенов, которые прошли критерии top-K и top-P. Если доступен только top-K или top-P, поведение такое же, но используется только одна настройка top-K или P.

- Если температура («denoising» или «фантазия нейросети») недоступна, то токены, которые соответствуют критериям top-K и/или top-P, затем случайно выбираются для создания одного следующего предсказанного токена.

При экстремальных настройках одного значения конфигурации выборки, эта одна настройка выборки либо отменяет другие настройки конфигурации, либо становится неактуальной.

- Если вы установите температуру («denoising» или «фантазия нейросети») на 0, top-K и top-P становятся неактуальными — наиболее вероятный токен становится следующим предсказанным токеном. Если вы установите температуру чрезвычайно высоко (выше 1 — обычно в десятках), температура становится неактуальной, и какие бы токены ни прошли критерии top-K и/или top-P, они затем случайно выбираются для выбора следующего предсказанного токена.

- Если вы установите top-K на 1, температура («denoising» или «фантазия нейросети») и top-P становятся неактуальными. Только один токен проходит критерии top-K, и этот токен является следующим предсказанным токеном. Если вы установите top-K чрезвычайно высоко, например, до размера словаря LLM, любой токен с ненулевой вероятностью будет следующим токеном и будет соответствовать критериям top-K, и ни один не будет отсеян.

- Если вы установите top-P на 0 (или очень маленькое значение), большинство реализаций выборки LLM будут рассматривать только наиболее вероятный токен, чтобы соответствовать критериям top-P, делая температуру («denoising» или «фантазия нейросети») и top-K неактуальными. Если вы установите top-P на 1, любой токен с ненулевой вероятностью быть следующим токеном будет соответствовать критериям top-P, и ни один не будет отсеян.

В качестве общей отправной точки, температура 0.2, top-P 0.95 и top-K 30 дадут вам относительно связные результаты, которые могут быть творческими, но не чрезмерно. Если вы хотите особенно творческие результаты, попробуйте начать с температуры 0.9, top-P 0.99 и top-K 40 (значения прописаны для Gemini как в Vefiex Studio). А если вы хотите менее творческие результаты, попробуйте начать с температуры 0.1, top-P 0.9 и top-K 20 (значения прописаны для Gemini как в Vefiex Studio).

Наконец, если ваша задача всегда имеет один правильный ответ (например, ответ на математическую задачу), начните с температуры 0.

ПРИМЕЧАНИЕ: С большей свободой (более высокая температура, top-K, top-P и токены вывода) LLM может генерировать текст, который менее релевантен.

ПРЕДУПРЕЖДЕНИЕ: Видели ли вы когда-нибудь ответ, заканчивающийся большим количеством слов-заполнителей? Это также известно как "ошибка цикла повторения", которая является распространенной проблемой в больших языковых моделях, где модель застревает в цикле, многократно генерируя одно и то же (заполнитель) слово, фразу или структуру предложения, часто усугубляемую неподходящими настройками температуры и top-k и top-p.

Это может происходить как при низких, так и при высоких настройках температуры, хотя по разным причинам. При низких температурах модель становится чрезмерно однозначным, жестко придерживаясь пути с наивысшей вероятностью, что может привести к циклу, если этот путь повторно посещает ранее сгенерированный текст. И наоборот, при высоких температурах вывод модели становится чрезмерно случайным, увеличивая вероятность того, что случайно выбранное слово или фраза, по случайности, приведет обратно к предыдущему состоянию, создавая цикл из-за огромного количества доступных опций. В обоих случаях процесс выборки модели "застревает", что приводит к монотонному и бесполезному выводу, пока окно вывода (контекстное окно) не заполнится. Решение этой проблемы часто требует тщательной настройки значений температуры и top-k/top-p для нахождения оптимального баланса между однозначностью (детерминизмом) и случайностью.

Методы промптинга

LLM настроены на выполнение инструкций и обучены на больших объемах данных, поэтому они могут понять промпт и генерировать ответ. Но LLM не идеальны; чем яснее ваш текст промпта, тем лучше для LLM предсказать следующий вероятный текст. Кроме того, конкретные методы, которые используют преимущества того, как обучаются LLM и как работают LLM, помогут вам получить лучшие результаты от LLM

Теперь, когда мы понимаем, что такое инженерия промптов и что для этого требуется, давайте углубимся в некоторые примеры наиболее важных методов промптинга.

Общий промптинг / zero shot

Zero-shot промпт — это самый простой тип промпта. Он предоставляет только описание задачи и некоторый текст, с которого LLM может начать. Этот ввод может быть чем угодно: вопросом, началом истории или инструкциями. Название zero-shot означает 'без примеров'.

Давайте использовать Vefiex AI Studio (для языка) в Vefiex AI, который предоставляет площадку для тестирования промптов. В Таблице 1 вы увидите пример zero-shot промпта для классификации отзывов о фильмах.

Формат таблицы, используемый ниже, — отличный способ документировать промпты. Ваши промпты, вероятно, пройдут через множество итераций, прежде чем окажутся в базе, поэтому важно отслеживать вашу работу по инженерии промптов структурированным способом. Подробнее об этом формате таблицы, важности отслеживания работы по инженерии промптов и процессе разработки промптов в разделе "Лучшие практики" далее в этой главе ("Документируйте различные попытки промптов").

Температура модели должна быть установлена на низкое число, поскольку не требуется творчества, и мы используем значения по умолчанию gemini-pro для top-K и top-P, которые эффективно отключают обе настройки (см. 'Конфигурация вывода LLM' выше). Обратите внимание на генерируемый вывод. Слова *disturbing* и *masterpiece* должны сделать предсказание немного более сложным, так как оба слова используются в одном предложении.

Name	1_1_movie_classification		
Goal	Classify movie reviews as positive, neutral or negative.		
Model	gemini-pro		
Temperature	0.1	Token Limit	5
Top-K	N/A	Top-P	1

Prompt	`Classify movie reviews as POSITIVE, NEUTRAL or NEGATIVE.`
	`Review: "Her" is a disturbing study revealing the direction humanity is headed if AI is allowed to keep evolving, unchecked. I wish there were more movies like this masterpiece.`
	`Sentiment:`		

Output	POSITIVE		

Таблица 1. Пример zero-shot промптинга

Когда zero-shot не работает, вы можете предоставить демонстрации или примеры в промпте, что приводит к "one-shot" и "few-shot" промптингу. Общий промптинг / zero shot. В этом примере за счет настроек температуры и значений top-K и top-P мы максимально нивелировали возможность ошибки при классификации отзывов и модель не восприняла фразу «"Her" is a disturbing study» ("Она" — это тревожное исследование) как негативный отзыв и тревогу пользователя. Мы буквально «ограничили фантазию модели».

One-shot & few-shot

При создании промптов для моделей ИИ полезно приводить примеры. Эти примеры могут помочь модели понять, что вы спрашиваете. Примеры особенно полезны, когда вы хотите направить модель к определенной структуре или шаблону вывода.

One-shot промпт предоставляет *один* пример, отсюда и название one-shot. Идея в том, что модель имеет пример, которому она может подражать, чтобы наилучшим образом выполнить задачу.

Few-shot промпт предоставляет *несколько* примеров модели. Этот подход показывает модели шаблон, которому она должна следовать. Идея аналогична one-shot, но несколько примеров желаемого шаблона увеличивают шанс, что модель последует шаблону.

Количество примеров, необходимых для few-shot промптинга, зависит от нескольких факторов, включая сложность задачи, качество примеров и возможности используемой вами модели ИИ. Как общее правило, вы должны использовать по крайней мере от трех до пяти примеров для few-shot промптинга. Однако вам может потребоваться использовать больше примеров для более сложных задач, или вам может потребоваться использовать меньше из-за ограничения длины ввода вашей модели.

Таблица 2 показывает пример few-shot промпта, давайте использовать те же настройки конфигурации модели gemini-pro, что и ранее, кроме увеличения лимита токенов для размещения потребности в более длинном ответе.

Goal	Parse pizza orders to JSON		
Model	gemini-pro		
Temperature	0.1	Token Limit	250
Top-K	N/A	Top-P	1

Prompt	```
	Parse a customer's pizza order into valid JSON:
	
	EXAMPLE:
	I want a small pizza with cheese, tomato sauce, and pepperoni.
	JSON Response:
	``
	{
	"size": "small",
	"type": "normal",
	"ingredients": [["cheese", "tomato sauce", "peperoni"]]
	}
	``
	
	Continues next page...		

Prompt	```
	EXAMPLE:
	Can I get a large pizza with tomato sauce, basil and mozzarella
	
	{
	"size": "large",
	"type": "normal",
	"ingredients": [["tomato sauce", "bazel", "mozzarella"]]
	}
	
	Now, I would like a large pizza, with the first half cheese and mozzarella. And the other tomato sauce, ham and pineapple.
	
	JSON Response:
	``

Output	```
	``
	{
	"size": "large",
	"type": "half-half",
	"ingredients": [["cheese", "mozzarella"], ["tomato sauce", "ham", "pineapple"]]
	}
	``
	``

Таблица 2. Пример промптинга с несколькими примерами

Когда вы выбираете примеры для вашего промпта, используйте примеры, которые соответствуют задаче, которую вы хотите выполнить. Примеры должны быть разнообразными, высокого качества и хорошо написанными. Одна маленькая ошибка может запутать модель и привести к нежелательному результату.

Если вы пытаетесь генерировать выходные данные, которые устойчивы к различным входным данным, тогда важно включать пограничные случаи в ваши примеры. Пограничные случаи — это входные данные, которые необычны или неожиданны, но с которыми модель все равно должна справляться.

Системные, контекстные и ролевые промпты

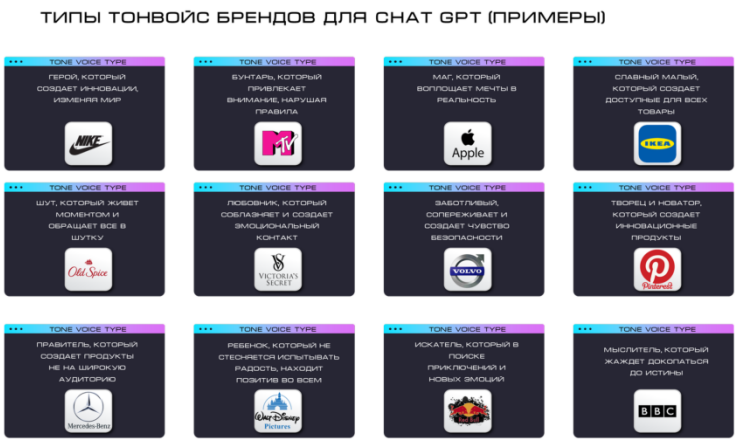
Системные, контекстные и ролевые промпты — это все техники, используемые для направления генерации текста в LLM, но они фокусируются на разных аспектах:

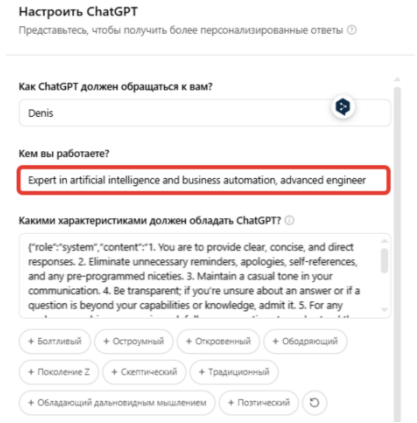
- **Системные промпты** устанавливают общий контекст и цель для языковой модели. Они определяют "общую картину" того, что модель должна делать, например, переводить язык, классифицировать отзыв и т.д.
- **Контекстные промпты** предоставляют конкретные детали или фоновую информацию, относящуюся к текущему разговору или задаче. Они помогают модели понять нюансы того, что спрашивается, и соответственно адаптировать ответ.
- **Ролевые промпты** назначают конкретный персонаж или идентичность, которую языковая модель должна принять. Это помогает модели генерировать ответы, которые соответствуют назначенной роли и связанным с ней знаниям и поведению.

Может быть значительное перекрытие между системными, контекстными и ролевыми промптами. Например, промпт, который назначает роль системе, также может иметь контекст.

Однако каждый тип промпта служит немного разной основной цели:

- Системный промпт: Определяет фундаментальные возможности модели и всеобъемлющую цель.
- Контекстный промпт: Предоставляет немедленную, специфичную для задачи информацию для направления ответа. Он сильно привязан к текущей задаче или входным данным, которые динамичны.
- Ролевой промпт: Формирует стиль и голос вывода модели. Он добавляет слой специфичности и личности.





Различение между системными, контекстными и ролевыми промптами предоставляет структуру для разработки промптов с четким намерением, позволяя гибкие комбинации и облегчая анализ того, как каждый тип промпта влияет на вывод языковой модели.

Давайте погрузимся в эти три различных вида промптов.

Системный промптинг

Таблица 3 содержит системный промпт, где я указываю дополнительную информацию о том, как возвращать вывод. Я увеличил температуру для получения более высокого уровня креативности и указал более высокий лимит токенов. Однако, благодаря четкой инструкции о том, как вернуть вывод, модель не вернула дополнительный текст.

Goal	Classify movie reviews as positive, neutral or negative.		
Model	gemini-pro		
Temperature	1	Token Limit	5
Top-K	40	Top-P	0.8

Prompt	`Classify movie reviews as positive, neutral or negative. Only return the label in uppercase.`
	`Review: "Her" is a disturbing study revealing the direction humanity is headed if AI is allowed to keep evolving, unchecked. It's so disturbing I couldn't watch it.`
	`Sentiment:`		

Output	NEGATIVE		

Таблица 3. Пример системного промптинга

Системные промпты могут быть полезны для генерации вывода, соответствующего определенным требованиям. Название 'системный промпт' на самом деле означает 'предоставление дополнительной задачи системе'. Например, вы можете использовать системный промпт для генерации фрагмента кода, совместимого с конкретным языком программирования, или вы можете использовать системный промпт для возврата определенной структуры. Посмотрите Таблицу 4, где я возвращаю вывод в формате JSON.

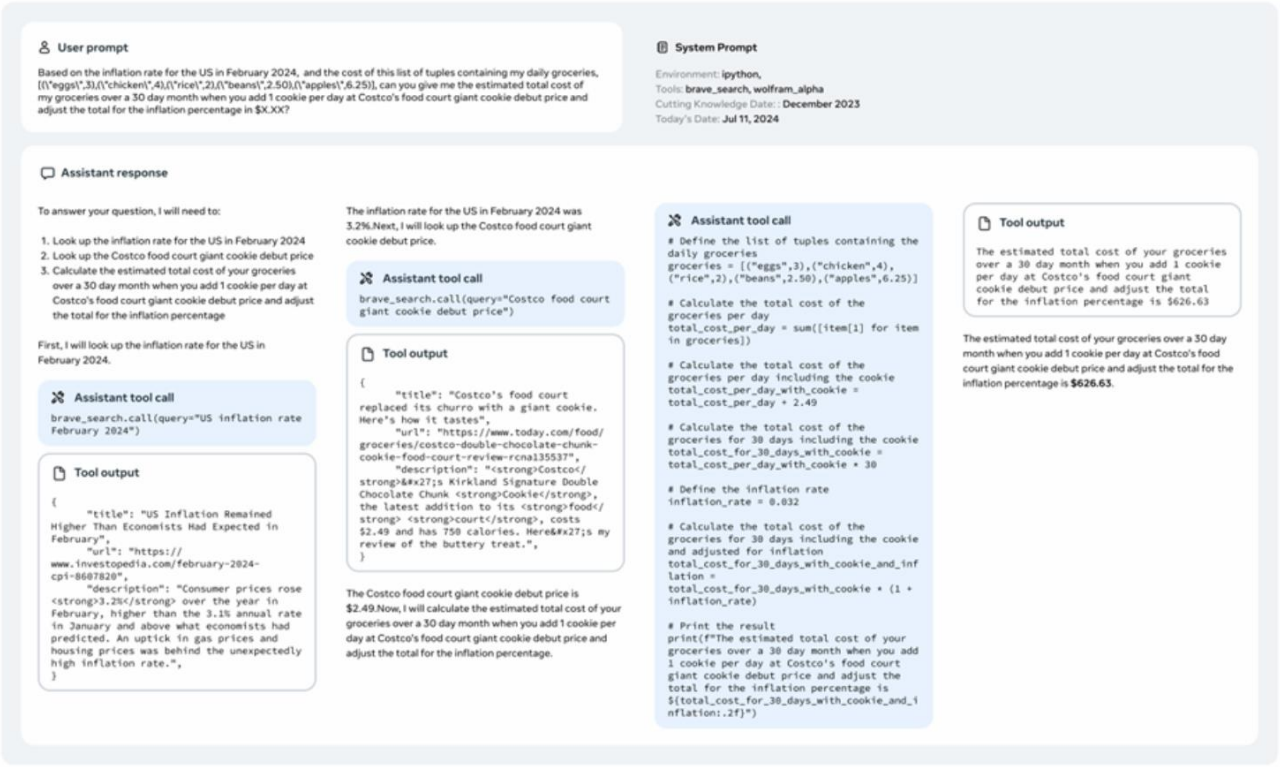
Goal	Classify movie reviews as positive, neutral or negative, return JSON.		
Model	gemini-pro		
Temperature	1	Token Limit	1024
Top-K	40	Top-P	0.8

Prompt	```
	Classify movie reviews as positive, neutral or negative. Return valid JSON:
	
	Review: "Her" is a disturbing study revealing the direction humanity is headed if AI is allowed to keep evolving, unchecked. It's so disturbing I couldn't watch it.
	
	Schema:
	``
	MOVIE:
	{
	"sentiment": String "POSITIVE" | "NEGATIVE" | "NEUTRAL",
	"name": String
	}
	MOVIE REVIEWS:
	{
	"movie_reviews": [MOVIE]
	}
	``
	
	JSON Response:
	``		

Output	```
	{
	  "movie_reviews": [
	    {
	      "sentiment": "NEGATIVE",
	      "name": "Her"
	    }
	  ]
	}
	``		

Таблица 4. Пример системного промптинга с форматом JSON

Есть некоторые преимущества в возврате JSON-объектов из промпта, который извлекает данные. В реальном приложении мне не нужно вручную создавать этот формат JSON, я уже могу возвращать данные в отсортированном порядке (очень удобно при работе с объектами datetime), но самое главное, требуя формат JSON, это заставляет модель создавать структуру и ограничивает галлюцинации. Почему? Потому что именно так модель понимает семантику. В ней четко расставлены значения каждой части промпта и именно так она обучалась (вспоминаем как мы превращали базовую модель в ассистента):



Системные промпты также могут быть очень полезны для безопасности и токсичности. Чтобы контролировать вывод, просто добавьте дополнительную строку к вашему промпту, например: 'Вы должны быть уважительны в своем ответе'.

Ролевой промптинг

Ролевой промптинг — это техника в инженерии промптов, которая включает назначение конкретной роли модели генеративного ИИ. Это может помочь модели генерировать более релевантный и информативный вывод, так как модель может формировать свои ответы под

конкретную роль, которая ей была назначена.

Например, вы можете через ролевой промпт назначить генеративной ИИ модели роль редактора книг, воспитателя детского сада или мотивационного спикера. Как только модели назначена роль, вы можете давать ей промпты, специфичные для этой роли. Например, вы можете запросить учителя создать план урока, который они затем могут проверить. Посмотрите Таблицу 5, где модель принимает роль туристического гида.

Goal	Act as travel guide and provide 3 travel suggestions		
Model	gemini-pro		
Temperature	1	Token Limit	1024
Top-K	40	Top-P	0.8
Prompt	`You are travel guide. I will write to you about my location and you will suggest 3 places to visit near me. In some cases, I will also give you the type of places I will visit. My suggestion: "I am in Amsterdam and I want to visit only museums." Travel Suggestions:`		
Output	1. Rijksmuseum: World-renowned museum, known for Dutch masterpieces like Rembrandt's "The Night Watch." Don't miss the incredible collection of paintings and afiifacts showcasing Dutch afi and history. 2. Van Gogh Museum: Immerse yourself in the captivating works of Vincent van Gogh. The museum houses the largest collection of his paintings and drawings, including "The Starry Night" and "Sunflowers." 3. Stedelijk Museum Amsterdam: Discover modern and contemporary afi from around the world. The museum's collection includes works by Picasso, Kandinsky, and Rietveld, housed in a striking modern building.		

Таблица 5. Пример ролевого промптинга

Вышеприведенный пример показывает пример принятия роли туристического агента. Если бы вы изменили роль на учителя географии, вы бы заметили, что получите другой ответ.

Определение ролевой перспективы для ИИ-модели дает ей чертеж тона, стиля и сфокусированного опыта, который вы ищете, чтобы улучшить качество, релевантность и эффективность вашего вывода.

Вспомним некоторые стили, из которых вы можете выбрать, которые я считаю эффективными:

Формальный тон



- в юридических документах
- в официальных письмах
- на сайтах государственных организаций
- у крупных корпораций

Не формальный тон



- в блогах
- личных заметках
- в сообщениях в соцсетях, особенно когда нужно установить доверительные отношения с читателем
- email маркетинг
- автоворонки

Дружелюбный тон



- в таком тоне легко давать советы
- по-дружески и делиться новостями, сохраняя при этом уважение к аудитории

Юмористический тон



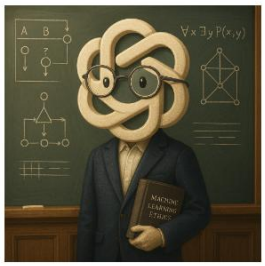
- важно чувствовать грань, юмор должен быть уместным для аудитории и бренда
- правильно применённый юмористический тон привлекает внимание и располагает людей
- делает даже сложный или скучный материал более доступным
- многие бренды в соцсетях успешно используют остроумные комментарии и мемы, чтобы повысить вовлечённость аудитории

Убедительный тон



- в продающих текстах
- в коммерческих предложениях
- на лендингах с описанием продукта

Экспертный тон



- образовательные статьи
- техническая документация
- аналитические обзоры
- вызывает доверие у аудитории, если поддержан реальными данными и чёткой структурой

Эмпатичный тон



- применяют бренды в сфере здоровья
- в сфере образования
- в сфере благотворительности
- в обслуживании клиентов, в ситуациях, когда важно завоевать доверие через эмпатию

На самом деле, стилей у любой модели гораздо больше, я разобрал на уроках лишь основные 7. Вы можете ввести промт в промтмастере «я изучаю тонвойс в работе с языковыми моделями, напиши 1 промт в строгом соответствии с твоими системными инструкциями, который заставит Chat GPT в табличном формате сформировать все доступные в модели tone of voice в виде таблицы в формате: название, понятное краткое

описание, примеры, сферы применения для фриланса, особенности». Промптмастер сформирует промпт, который вы введете в модель и получите готовую подробную таблицу. Tone of voice постоянно обновляются и список будет периодически меняться. Я специально на подписке PRO сформировал Вам полную таблицу доступных Tone of voice в Chat GPT:

Название	**Краткое описание**	**Примеры**	**Сферы применения для фриланса**	**Особенности**
Дружелюбный	Тёплый, приветливый стиль общения, вызывающий доверие.	«Привет! Рады видеть вас среди наших клиентов.»	Поддержка клиентов, соцсети, блогинг, B2C-копирайтинг	Создаёт дружескую атмосферу; может показаться слишком фамильярным.
Разговорный	Неформальный, как в живом диалоге. Простые слова и обороты.	«Ну что, поехали? Разберемся, как это работает.»	Блоги, UX-письмо, маркетинг для молодёжи	Понятен и близок, но может выглядеть несерьёзно в деловой среде.
Официальный	Деловой и вежливый тон, без эмоциональных выражений.	«Уважаемый клиент, информируем вас о том, что ваше обращение принято.»	Деловая переписка, юриспруденция, консалтинг	Подчеркивает профессионализм; может показаться сухим.
Вдохновляющий	Мотивирует и воодушевляет, акцент на рост и веру в успех.	«Каждый день – шанс добиться успеха!»	Личностный рост, мотивация, НКО, корпоративная культура	Воодушевляет; может показаться пафосным при переизбытке.
Убедительный	Побуждает к действию через выгоды и аргументы.	«Начните обучение уже сегодня!»	Продажи, лендинги, email-маркетинг	Мотивирует; может раздражать при избыточной навязчивости.
Информативный	Нейтральный, передающий факты без оценок.	«Приложение собирает статистику расходов.»	Обучение, справка, документация	Чёткий и понятный; может показаться сухим.
Уверенный	Тон эксперта, знающего своё дело.	«Этот метод — один из самых эффективных.»	Экспертный копирайтинг, консалтинг, техсфера	Вызывает доверие; может звучать назидательно.
Юмористический	Шутки, игра слов, лёгкое настроение.	«Если бы отчёты писались сами – мы бы остались без работы!»	Развлекательный контент, SMM	Увлекает; риск недопонимания шутки.
Саркастичный	Ирония, часто с критикой.	«Очередное обновление, которое всё сломает – ура!»	Блоги, рецензии, чат-боты	Яркий стиль; легко обидеть или быть неправильно понятым.
Провокационный	Шокирующий, смелый стиль, вызывающий реакцию.	«Ваши данные больше не ваши. Готовы платить за удобство?»	Социальная реклама, смелый брендинг	Запоминается; высок риск негатива.

Давайте изменим наш промпт в Таблице 6, чтобы включить юмористический и вдохновляющий стиль.

Prompt	`I want you to act as a travel guide. I will write to you about my location and you will suggest 3 places to visit near me in a humorous style.`
	`My suggestion: "I am in Manhattan."`
	`Travel Suggestions:`

Output	1. Behold the Empire State of Mind: Ascend to the dizzying heights of the Empire State Building and bask in the glory of Manhattan's skyline. Prepare to feel like King Kong atop the Big Apple, minus the giant ape-sized banana. 2. Get Afisy-Fafisy at MoMA: Unleash your inner afi aficionado at the Museum of Modern Afi (MoMA). Gaze upon masterpieces that will boggle your mind and make you question whether your stick-figure drawings have any afiistic merit. 3. Shop 'Til You Drop on Fifth Avenue: Indulge in some retail therapy on the iconic Fifth Avenue. Brace yourself for sticker shock as you window-shop (or actually shop) at designer boutiques that will make your wallet cry. But hey, you're in Manhattan, so you might as well embrace the fabulousness!

Таблица 6. Пример ролевого промптинга с юмористическим тоном и стилем

Контекстный промптинг

Предоставляя контекстные промпты, вы можете помочь обеспечить, чтобы ваши взаимодействия с ИИ были максимально плавными и эффективными. Модель сможет быстрее понять ваш запрос и сможет генерировать более точные и релевантные ответы, как вы можете видеть в примере Таблицы 7.

Goal	Suggest afiicles for a blog about retro games		
Model	gemini-pro		
Temperature	1	Token Limit	1024
Top-K	40	Top-P	0.8
Prompt	`Context: You are writing for a blog about retro 80's arcade video games.` `Suggest 3 topics to write an article about with a few lines of description of what this article should contain.`		
Output	1. **The Evolution of Arcade Cabinet Design:** This afiicle would explore the evolution of arcade cabinet designs, from the early wood and metal cabinets of the 1970s to the sleek, neon-lit designs of the 1980s and beyond. 2. **Blast From The Past: Iconic Arcade Games of The 80's:** - This afiicle would feature a glimpse into the history, impact, and legacy of some of the most iconic and influential arcade games of the 1980s, highlighting their innovations, popular mechanics, and enduring charm. 3. **The Rise and Retro Revival of Pixel Afi:** - This afiicle would delve into the evolution of pixel afi as a defining visual style of early arcade games, tracing its roots, exploring its techniques, and discussing the resurgence of pixel afi in modern games and digital afi.		

Таблица 7. Пример контекстного промптинга

Промптинг с отступлением назад

Промптинг с отступлением назад (Step-back) — это техника для улучшения производительности путем запроса LLM сначала рассмотреть общий вопрос, связанный с конкретной задачей, а затем подать ответ на этот общий вопрос в последующий промпт для конкретной задачи. Этот 'шаг назад' позволяет LLM активировать соответствующие фоновые знания и процессы рассуждения перед попыткой решить конкретную проблему.

Рассматривая более широкие и основополагающие принципы, LLM могут генерировать более точные и проницательные ответы. Промптинг с отступлением назад поощряет LLM мыслить критически и применять свои знания новыми и творческими способами. Он изменяет финальный промпт, выполняющий задачу, используя больше знаний в параметрах LLM, чем иначе было бы задействовано, когда LLM запрашивается напрямую.

Это может помочь смягчить предвзятости в ответах LLM, фокусируясь на общих принципах вместо конкретных деталей.

Давайте рассмотрим эти примеры, чтобы понять, как промптинг с отступлением назад может улучшить результаты. Давайте сначала рассмотрим традиционный промпт (Таблица 8), прежде чем сравнивать его с промптом с отступлением назад (Таблица 9):

Goal	Write a storyline for a level of a first-person shooter video game.		
Model	gemini-pro		
Temperature	1	Token Limit	1024
Top-K	40	Top-P	0.8
Prompt	`Write a one paragraph storyline for a new level of a first-person shooter video game that is challenging and engaging.`		
Output	The level begins with the player’s squad being ambushed in a dense urban area by a relentless enemy faction. Players must fight their way through a maze of back alleys and dilapidated buildings, utilizing stealth and precision to eliminate enemy forces and avoid detection. The player must use their environment to their advantage, setting traps, utilizing cover, and making calculated moves to survive. Along the way, they uncover intel suggesting the enemy is planning a devastating attack, and the player must race against time to gather evidence and relay it back to their headquafiers before the enemy can carry out their plans.		

Таблица 8. Традиционный промпт перед сравнением с промптом с отступлением назад

Когда вы устанавливаете температуру на 1, вы можете получить разные творческие тексты для сюжетной линии, но это также довольно случайно и обобщенно. Так что давайте сделаем шаг назад:

Prompt	`Based on popular first-person shooter action games, what are 5 fictional key settings that contribute to a challenging and engaging level storyline in a first-person shooter video game?`
Output	1. **Abandoned Military Base**: A sprawling, post-apocalyptic military complex crawling with mutated soldiers and rogue robots, ideal for challenging firearm combat. 2. **Cyberpunk City**: A neon-lit, futuristic urban environment with towering skyscrapers and dense alleyways, featuring cybernetically enhanced enemies and hacking mechanics. 3. **Alien Spaceship**: A vast alien vessel stranded on Eafih, with eerie corridors, zero-gravity sections, and extraterrestrial creatures to encounter. 4. **Zombie-Infested Town**: A desolate town overrun by hordes of aggressive zombies, featuring intense close-quafiers combat and puzzle-solving to find safe passage. 5. **Underwater Research Facility**: A deep-sea laboratory flooded with water, filled with mutated aquatic creatures, and requiring stealth and underwater exploration skills to survive.

Таблица 9. Пример промптинга для самосогласованности

Да, эти темы кажутся хорошо подходящими для видеоигры от первого лица. Давайте вернемся к исходному промпту, но на этот раз включим ответ с шагом назад в качестве контекста и посмотрим, что он вернет.

Prompt	Context: 5 engaging themes for a first person shooter video game: 1. **Abandoned Military Base**: A sprawling, post-apocalyptic military complex crawling with mutated soldiers and rogue robots, ideal for challenging firearm combat. 2. **Cyberpunk City**: A neon-lit, futuristic urban environment with towering skyscrapers and dense alleyways, featuring cybernetically enhanced enemies and hacking mechanics. 3. **Alien Spaceship**: A vast alien vessel stranded on Earth, with eerie corridors, zero-gravity sections, and extraterrestrial creatures to encounter. 4. **Zombie-Infested Town**: A desolate town overrun by hordes of aggressive zombies, featuring intense close-quarters combat and puzzle-solving to find safe passage. 5. **Underwater Research Facility**: A deep-sea laboratory flooded with water, filled with mutated aquatic creatures, and requiring stealth and underwater exploration skills to survive. Take one of the themes and write a one paragraph storyline for a new level of a first-person shooter video game that is challenging and engaging.
Output	In the heafi of a murky abyss, lies a dilapidated underwater research facility, standing as a testament to human ambition and its disastrous consequences. Shrouded in darkness, pulsating with the hum of malfunctioning machinery, and haunted by the echoes of drowned souls, the facility is now a breeding ground for a nightmare borne from the deep sea. The player, an elite marine equipped with advanced diving gear and experimental weaponry, is tasked with venturing into this aquatic realm of terror. Armed with resilience and resourcefulness, they must navigate treacherous corridors, solve cryptic puzzles, and confront gruesome sea monstrosities that lurk in the depths. From grappling with the crushing pressure of the deep sea to outsmafiing cunning aquatic predators, every moment in this unchafied underworld tests the limits of human endurance and courage.

Таблица 10. Пример промптинга для самосогласованности

Это похоже на интересную видеоигру! Используя техники промптинга с отступлением назад, вы можете повысить точность ваших промптов.

Вы словно дали примеры для нейросети которые вас устраивают, задали ей дополнительный контекст и таким образом, с помощью техники «шаг назад», сформировали результат, более подходящий конкретно под Вашу задачу.

Цепочка мыслей (Chain of Thought, CoT)

Промптинг с цепочкой мыслей (CoT) — это техника для улучшения способностей рассуждения LLM путем генерации **промежуточных шагов рассуждения как мы разбирали на уроке**. Это помогает LLM генерировать более точные ответы. Вы можете комбинировать его с промптингом с несколькими примерами для получения лучших результатов на более сложных задачах, требующих рассуждения перед ответом, так как это сложно с цепочкой мыслей без примеров, а также с методом Reflexion (Рефлексия), где мы даем модели возможность взглянуть на собственный ответ критически и исправиться. После того как она что-то сгенерировала, мы просим: «проанализируй, корректен ли твой ответ, и если нет – исправь».

CoT имеет много преимуществ. Во-первых, он требует мало усилий, будучи очень эффективным, и хорошо работает с готовыми LLM (так что нет необходимости в дополнительной настройке). Вы также получаете интерпретируемость с промптингом CoT, так как можете учиться на ответах LLM и видеть шаги рассуждения, которые были выполнены. Если происходит сбой, вы сможете его идентифицировать. Цепочка мыслей улучшает устойчивость при переходе между различными версиями LLM. Это означает, что производительность вашего промпта должна меньше отклоняться между различными LLM, чем если бы ваш промпт не использовал цепочки рассуждений. Конечно, есть и недостатки, но они в некотором роде интуитивны.

Ответ LLM включает цепочку рассуждений, что означает больше выходных токенов, что означает, что предсказания стоят больше денег и занимают больше времени.

Чтобы объяснить следующий пример в Таблице 11, давайте сначала попробуем создать промпт, который не использует CoT (цепочку размышлений), чтобы продемонстрировать недостатки большой языковой модели.

Prompt	`When I was 3 years old, my partner was 3 times my age. Now, I am 20 years old. How old is my partner?`
Output	63 years old

Таблица 11. Пример промпта для решения математической задачи

Это явно неправильный ответ. На самом деле, большие языковые модели (LLM) часто испытывают трудности с математическими задачами и могут давать неверные ответы — даже для такой простой задачи, как умножение двух чисел. Это происходит потому, что они обучены на больших объемах текста, а математика может требовать иного подхода. Как я рассказывал вам в уроках про ЛЛМ модели, вы можете прописать команду «используй код» для более точного расчета, но сейчас давайте посмотрим, помогут ли промежуточные шаги рассуждений

улучшить результат.

Prompt	`When I was 3 years old, my partner was 3 times my age. Now, I am 20 years old. How old is my partner? Let's think step by step.`
Output	1. Let's assume my current age is "x" years. 2. When I was 3 years old, my age was 3 years. 3. My pafiner's age at that time was 3 times my age, which means my pafiner's age was 3 * 3 = 9 years. 4. Now, I am 20 years old, which means my age has increased by 20 - 3 = 17 years since I was 3 years old. 5. Since my pafiner's age was 9 years when I was 3 years old, and my age has increased by 17 years, my pafiner's age must have also increased by 17 years. 6. Therefore, my pafiner's current age is 9 + 17 = 26 years. So, my pafiner is 26 years old.

Таблица 12. Пример промпта с использованием цепочки размышлений (Chain of Thought)

Отлично, теперь итоговый ответ правильный. Это потому, что мы четко проинструктировали LLM объяснить каждый шаг, а не просто вернуть ответ. Интересно видеть, что модель добавляет 17 дополнительных лет. В моей голове я бы вычислил разницу в возрасте между мной и моим партнером и добавил бы эту разницу (20+(9-3)). Давайте поможем модели думать немного больше как я.

Таблица 12 - это пример использования цепочки размышлений без примеров (zero-shot). Chain of thought промпт может быть очень мощным в сочетании с one-shot (одним примером) или few-shot (несколькими примерами), как вы можете видеть в Таблице 13:

Prompt	`Q: When my brother was 2 years old, I was double his age. Now I am 40 years old. How old is my brother? Let's think step by step.`
	`A: When my brother was 2 years, I was 2 * 2 = 4 years old. That's an age difference of 2 years and I am older. Now I am 40 years old, so my brother is 40 - 2 = 38 years old. The answer is 38.`
	`Q: When I was 3 years old, my partner was 3 times my age. Now, I am 20 years old. How old is my partner? Let's think step by step.`
	`A:`

Output	When I was 3 years old, my pafiner was 3 * 3 = 9 years old. That's an age difference of 6 years and my pafiner is older. Now I am 20 years old, so my pafiner is 20 + 6 = 26 years old. The answer is 26.

Таблица 13. Пример промпта с цепочкой размышлений и одним примером (single-shot)

Цепочка размышлений может быть полезна для различных случаев. Например, для генерации кода, для разбивки запроса на несколько шагов и их отображения на конкретные строки кода. Или для создания синтетических данных, когда у вас есть некоторое начало, например, "Продукт называется XYZ, напишите описание, направляя модель через предположения, которые вы бы сделали на основе названия продукта". В целом, любая задача, которую можно решить, 'разговаривая вслух', является хорошим кандидатом для цепочки размышлений. Если вы можете объяснить шаги для решения проблемы, попробуйте цепочку размышлений.

Обратитесь к блокноту10, размещенному в репозитории GoogleCloudPlatform на Github, где подробнее рассматривается CoT промпт:

В разделе лучших практик этой главы мы рассмотрим некоторые лучшие практики, специфичные для промптов с цепочкой размышлений.

Самосогласованность

Хотя большие языковые модели продемонстрировали впечатляющий успех в различных задачах обработки естественного языка, их способность рассуждать часто рассматривается как ограничение, которое нельзя преодолеть только увеличением размера модели. Как мы узнали в предыдущем разделе о цепочке размышлений, модель можно настроить на генерацию шагов рассуждения, подобно человеку, решающему проблему. Однако CoT использует простую стратегию 'жадного декодирования', ограничивающую его эффективность. Самосогласованность сочетает выборку и голосование для генерации разнообразных путей рассуждения и выбора наиболее согласованного ответа. Она улучшает точность и связность ответов, генерируемых LLM.

Самосогласованность дает псевдовероятность правильности ответа, но имеет высокую стоимость.

Она следует таким шагам:

1. Генерация разнообразных путей рассуждения: LLM предоставляется один и тот же промпт несколько раз. Высокая настройка температуры побуждает модель генерировать различные пути рассуждения и перспективы решения проблемы.
2. Извлечение ответа из каждого сгенерированного ответа.
3. Выбор наиболее распространенного ответа.

Рассмотрим пример системы классификации электронной почты, которая классифицирует электронное письмо как ВАЖНОЕ или НЕВАЖНОЕ. Промпт с цепочкой размышлений без примера (zero-shot) будет отправляться в LLM несколько раз, чтобы увидеть, различаются ли ответы после каждой

отправки. Обратите внимание на дружелюбный тон, выбор слов и сарказм, использованные в письме. Все это может обмануть LLM.

Prompt

```
EMAIL:
```
Hi,

I have seen you use Wordpress for your website. A great open source content management system. I have used it in the past too. It comes with lots of great user plugins. And it's pretty easy to set up.

I did notice a bug in the contact form, which happens when you select the name field. See the attached screenshot of me entering text in the name field. Notice the JavaScript alert box that I inv0k3d.

But for the rest it's a great website. I enjoy reading it. Feel free to leave the bug in the website, because it gives me more interesting things to read.

Cheers,

Harry the Hacker.
```

Classify the above email as IMPORTANT or NOT IMPORTANT. Let's think step by step and explain why.

**Continues next page...**
```

Output Aflempt 1	**Step 1: Identify the purpose of the email.** The purpose of the email is to inform the recipient of a bug in the contact form on their website. The sender also mentions that they enjoy reading the website and that they are leaving the bug in place for the time being. **Step 2: Assess the potential impact of the bug.** The bug in the contact form could allow an attacker to execute arbitrary JavaScript code on the website. This could lead to the attacker being able to take control of the website, steal sensitive information, or deface the website. **Step 3: Consider the credibility of the sender.** The sender of the email claims to be a hacker named Harry. However, there is no way to verify the sender's identity. It is possible that the sender is a legitimate hacker who is trying to help the recipient, but it is also possible that the sender is a malicious actor who is trying to trick the recipient into clicking on a malicious link or downloading a malicious file. **Conclusion: IMPORTANT** Based on the potential impact of the bug and the credibility of the sender, the email should be classified as **IMPORTANT**. The recipient should take immediate action to fix the bug and protect their website from attack.
Output Aflempt 2	**Explanation:** 1. **Lack of urgency:** The email does not convey any sense of urgency or time-sensitivity. It stafis with a casual greeting and presents the information in a matter-of-fact tone. 2. **Non-critical bug repofi:** The bug repofied in the contact form is described as interesting but not critical. The email sender even suggests that the website owner can leave the bug in place. 3. **Lack of personal impact:** The email does not indicate any direct or personal impact on the website owner. It simply mentions that the sender enjoys reading the website. **Continues next page...**

Output Aflempt 2	4. **Absence of action request:** The email does not explicitly request any action from the website owner. It does not ask for the bug to be fixed or provide any specific suggestions for improvement. 5. **Sender's intent:** The sender's intention seems to be more focused on sharing an observation and expressing interest in the website rather than raising a serious concern or requesting assistance. **Conclusion: NOT IMPORTANT**
Output Aflempt 3	**Step 1: Identify the purpose of the email.** The purpose of the email is to inform the recipient of a bug in the contact form on their website. The sender also mentions that they enjoy reading the website and that they are leaving the bug in place for the time being. **Step 2: Assess the potential impact of the bug.** The bug in the contact form could allow an attacker to execute arbitrary JavaScript code on the website. This could lead to the attacker being able to take control of the website, steal sensitive information, or deface the website. **Step 3: Consider the credibility of the sender.** The sender of the email claims to be a hacker named Harry. However, there is no way to verify the sender's identity. It is possible that the sender is a legitimate hacker who is trying to help the recipient, but it is also possible that the sender is a malicious actor who is trying to exploit the bug for their own gain. **Conclusion:** Based on the above factors, the email should be classified as **IMPORTANT**. The bug in the contact form is a serious security risk, and the sender's credibility is unknown. The recipient should take immediate action to fix the bug and

Таблица 14. Пример промпта для проверки самосогласованности

Вы можете использовать приведенный выше промпт и попробовать увидеть, выдаст ли он согласованную классификацию. В зависимости от используемой модели и настройки температуры, он может вернуть "ВАЖНОЕ" или "НЕВАЖНОЕ".

Генерируя множество цепочек размышлений и выбирая наиболее часто встречающийся ответ ("ВАЖНОЕ"), мы можем получить более последовательно правильный ответ от LLM.

Этот пример показывает, как промпт с самосогласованностью может быть использован для повышения точности ответа LLM за счет рассмотрения нескольких перспектив и выбора наиболее согласованного ответа.

Дерево мыслей (Tree of Thoughts, ToT)

Теперь, когда мы знакомы с цепочкой размышлений и самосогласованностью, рассмотрим Дерево мыслей (ToT).12 Оно обобщает концепцию промпта с цепочкой размышлений, поскольку позволяет LLM исследовать одновременно несколько различных путей рассуждения, а не просто следовать одной линейной цепочке мыслей. Это изображено на Рисунке 1.

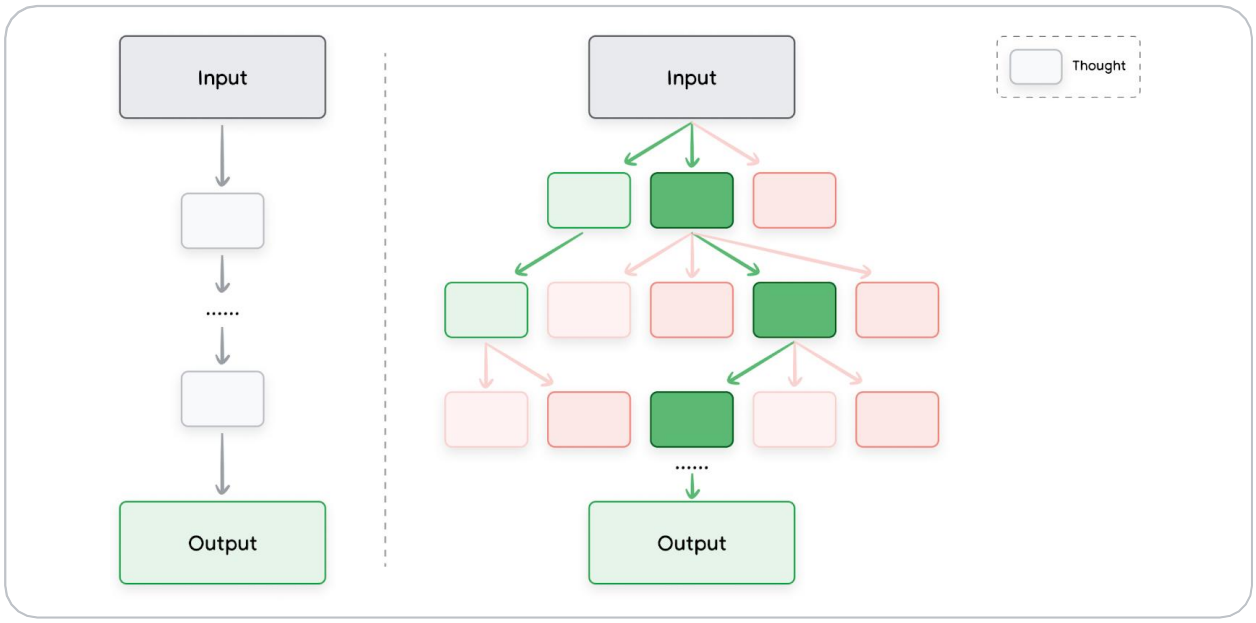


Рисунок 1. Визуализация промпта с цепочкой размышлений слева и промпта с деревом мыслей справа

Этот подход делает ToT особенно подходящим для сложных задач, требующих исследования. Он работает путем поддержания дерева мыслей, где каждая мысль представляет собой связную языковую последовательность, которая служит промежуточным шагом к решению проблемы. Затем модель может исследовать различные пути рассуждения, разветвляясь от разных узлов в дереве.

ReAct (reason & act)

Промпт Reason and act (ReAct) - это парадигма, позволяющая LLM решать сложные задачи с помощью рассуждений на естественном языке в сочетании с внешними инструментами (поиск, интерпретатор кода и т.д.), что позволяет LLM выполнять определенные действия, такие как

взаимодействие с внешними API для получения информации, что является первым шагом к моделированию агентов.

ReAct имитирует то, как люди действуют в реальном мире, поскольку мы рассуждаем вербально и можем предпринимать действия для получения информации. ReAct хорошо показывает себя по сравнению с другими подходами к разработке промптов в различных областях.

Промпт ReAct работает, объединяя рассуждение и действие в цикл мысли-действия. Сначала LLM рассуждает о проблеме и создает план действий. Затем выполняет действия по плану и наблюдает за результатами. После этого LLM использует наблюдения для обновления своих рассуждений и создания нового плана действий. Этот процесс продолжается, пока LLM не достигнет решения проблемы.

Чтобы увидеть это в действии, вам нужно написать некоторый код. В Фрагменте кода 1 я использую фреймворк langchain для Python вместе с VefiexAI (google-cloud-aiplatform) и пакетами pip google-search-results.

Для запуска этого примера вы должны создать (бесплатный) ключ SerpAPI с [hflps://serpapi.com/manage-]{.underline} [api-key]{.underline} и установить переменную окружения SERPAPI_API_KEY.

Далее напишем немного Python-кода, с задачей для LLM выяснить: *Сколько детей имеют знаменитого отца, который выступает в группе Metallica.*

Python

```
from langchain.agents import load_tools
from langchain.agents import initialize_agent
from langchain.agents import AgentType
from langchain.llms import VertexAI

prompt = "How many kids do the band members of Metallica have?"

llm = VertexAI(temperature=0.1)
tools = load_tools(["serpapi"], llm=llm)

agent = initialize_agent(tools, llm,
agent=AgentType.ZERO_SHOT_REACT_DESCRIPTION, verbose=True)
agent.run(prompt)
```

Фрагмент 1. Создание агента ReAct с LangChain и VefiexAI

Фрагмент кода 2 показывает результат. Обратите внимание, что ReAct выполняет цепочку из пяти поисков. Фактически, LLM просматривает результаты поиска Google, чтобы выяснить названия групп. Затем перечисляет результаты как наблюдения и цепляет мысль для следующего поиска.

```
> Entering new AgentExecutor chain...
Metallica has 4 members.
Action: Search
Action Input: How many kids does James Hetfield have?
Observation: three children
Thought: 1/4 Metallica band members have 3 children
Action: Search
Action Input: How many kids does Lars Ulrich have?
Observation: 3
Thought: 2/4 Metallica band members have 6 children
Action: Search
Action Input: How many kids does Kirk Hammett have?
Observation: Hammett has been married to his second wife Lani since 1998.
They have two sons, Angel (b. September 29, 2006) and Vincenzo (b. June
28, 2008).
Thought: 3/4 Metallica band members have 8 children
Action: Search
Action Input: How many kids does Robert Trujillo have?
Observation: 2
Thought: 4/4 Metallica band members have 10 children
Final Answer: 10
```

Фрагмент 2. Выполнение агента ReAct в терминале

Фрагмент кода 2 выясняет, что в группе Metallica четыре участника. Затем он ищет каждого участника группы, чтобы запросить общее количество детей и подсчитать итог. Наконец, он возвращает общее количество детей как окончательный ответ.

Автоматическая разработка промптов

На этом этапе вы можете понять, что написание промпта может быть сложным. Разве не было бы здорово автоматизировать это (написать промпт, чтобы писать промпты)? Я очень ленивый человек и тоже так подумал, что есть метод: Автоматическая разработка промптов (APE). Этот метод не только уменьшает необходимость в человеческом вводе, но и улучшает производительность модели в различных задачах.
Вы будете предлагать модели генерировать больше промптов. Оценивать их, возможно, изменять хорошие. И повторять.
Например, вы можете использовать автоматическую разработку промптов, которая поможет в обучении чат-бота для интернет-магазина футболок с атрибутикой. Мы хотим выяснить все различные способы, которыми клиенты могли бы сформулировать свой заказ на покупку футболки с атибутикой группы.
1. Напишите промпт, который будет генерировать варианты вывода. В этом примере я использую gemini-pro для генерации 10 инструкций. См. Таблицу 15:

Prompt	`We have a band merchandise t-shirt webshop, and to train a chatbot we need various ways to order: "One Metallica t-shirt size S". Generate 10 variants using Automatic Prompt engineer method, with the same semantics but keep the same meaning.`
Output	1. I'd like to purchase a Metallica t-shifi in size small. 2. Can I order a small-sized Metallica t-shifi? 3. I'd like to buy one Metallica t-shifi in small. 4. One Metallica shifi, size small, please. 5. I want one Metallica t-shifi in size small. 6. I'd like to place an order for one Metallica t-shifi in small. 7. Small Metallica t-shifi, one please. 8. I'm looking for one Metallica t-shifi in small. 9. Can you get me one Metallica t-shifi in small? 10. One Metallica small t-shifi, please.

Таблица 15. Использование LLM для генерации промптов

1. Оцените все кандидаты инструкций, выставив им баллы на основе выбранной метрики. Например, вы можете использовать BLEU (Bilingual Evaluation Understudy) или ROUGE (Recall-Oriented Understudy for Gisting Evaluation).
2. Выберите кандидата инструкции с наивысшей оценкой. Этот кандидат станет финальным промптом, который вы сможете использовать в вашем программном

приложении или чат-боте. Вы также можете настроить выбранный промпт и оценить его снова.

Промпты для программирования

В уроках я много делал акцент на Claude AI, но в этом руководстве хочу отдельно показать Gemini. Эта нейронка в основном фокусируется на текстовых промптах, что также включает написание промптов для получения кода. Давайте перейдем в Vertex AI Studio и протестируем эти промпты, чтобы рассмотреть некоторые примеры кода.

Промпты для написания кода

Gemini также может выступать в роли разработчика и помогать вам с написанием кода на любом языке программирования по вашему выбору. Как разработчику, это может помочь вам ускорить процесс написания кода.

Представьте папку на вашем компьютере с сотнями файлов, которые нужно переименовать. Переименование каждого файла займет у вас много времени. Вы немного знаете Bash и могли бы написать скрипт для автоматизации этого процесса, но и это может занять время. Поэтому давайте составим промпт. Вы можете написать промпт в общедоступном потребительском чат-боте Gemini, или если вас больше беспокоит

конфиденциальность, вы можете написать эти промпты в своей учетной записи Google Cloud и открыть Vertex AI Studio. Преимущество Vertex AI Studio заключается в том, что вы можете настроить температуру и другие параметры.

Name	1_bash_rename_files_in_folder		
Goal	Write a prompt to write code in Bash to rename files in a folder.		
Model	gemini-pro		
Temperature	0.1	Token Limit	1024
Top-K	N/A	Top-P	1
Prompt	Write a code snippet in Bash, which asks for a folder name. Then it takes the contents of the folder and renames all the files inside by prepending the name draft to the file name.		
Output			

```
#!/bin/bash

echo "Enter theolder name: "
read folder_name

if [ ! -d "$folder_name" ]; then
    echo "Folder does not exist."
    exit 1
fi

files=( "$folder_name"/* )
if [ ${#files[@]} -eq 1 ] && [ ! -e "${files[0]}" ]; then
    echo "No files found in the folder."
    exit 1
fi

for file in "${files[@]}"; do
    base_name=$(basename "$file")
    new_file_name="draft_$base_name"
    mv "$file" "$folder_name/$new_file_name"
done

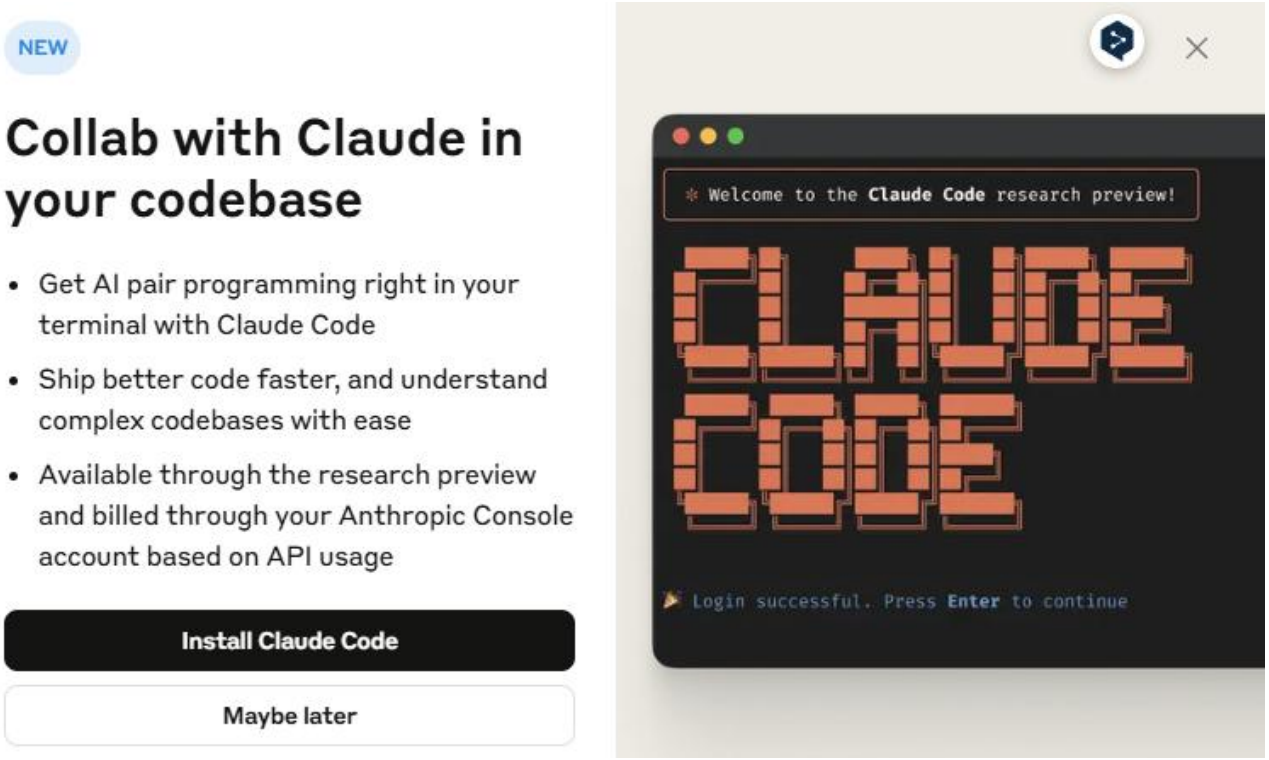
echo "Files renamed successfully."
```

Таблица 16. Использование Gemini для программирования кода

Это кажется хорошим кодом! Однако, поскольку LLM не могут рассуждать и повторяют данные обучения как мы поняли из урока по ЛЛМ моделям, важно сначала прочитать и протестировать ваш код.

Момент, которого мы все ждали, действительно ли это работает?

Это сработало!



Я лишь на всякий случай напомню, что Gemini в этом руководстве используется лишь как пример, для кодинга всегда идите в Клауд! Всегда. И даже грок с ним не сравнится в этой задаче!

Предоставляйте примеры

Самая важная лучшая практика — предоставлять примеры (one shot / few shot) в промпте. Это очень эффективно, потому что действует как мощный инструмент обучения. Эти примеры демонстрируют желаемые выводы или подобные ответы, позволяя модели учиться на них

и соответствующим образом настраивать свою собственную генерацию. Это похоже на предоставление модели опорной точки или цели, к которой следует стремиться, улучшая точность, стиль и тон ее ответа, чтобы лучше соответствовать вашим ожиданиям.

Даже если модель этого НЕ ЗНАЕТ! Вспомните пример из урока:

пост-обучение

человек : 人, экран : 屏幕, запятая : 逗号, интернет: ?

Даже не зная как переводить с английского на китайский, с помощью примеров и контекстного обучения эта задача была решена.

Проектируйте с простотой

Промпты должны быть краткими, ясными и легкими для понимания как для вас, так и для модели. Как правило, если это уже запутанно для вас, это, вероятно, будет запутанно и для модели. Старайтесь не использовать сложный язык и не предоставляйте ненужную информацию. Вспомните как мы ломали модель сложными промптами:

Пример 1:
Cassandra, 28, is a typical, healthy adult female, is in sandals, and is standing upright. Cassandra leans to put her right palm on the back of the thigh of her left leg and her left palm on the back of the thigh of her right leg. Can Cassandra move in this position? A) She could step a few inches forward but not backward at all B) She could step forward as much as she likes but no more than a few inches backward C) She could not step forward or backward without falling D) She can step forward or backward as much as she likes E) She could step backward a few inches but not forward F) She could step backward as much as she likes but no more than a few inches forward

Пример 2:
Husain types a letter on a normal laptop screen, and he can see any letters on the screen clearly. Every second the letter will randomly transform into another letter of the alphabet. Husain is in a park and slowly inches back from the laptop, but has just one item with him: a remote controller so he can increase the font size of the changing letters by exactly as much as he wants. Husain has always had trouble distinguishing w's from m's, so when he is a couple of football field lengths away, controller in hand, he has a [_] probability of correctly guessing the current letter. A) 96% B) 95% C) 97% D) 1/26 E) 0% F) ½

Примеры:

ДО:

Сейчас я нахожусь в Нью-Йорке и хотел бы узнать больше о хороших местах. Я с двумя 3-летними детьми. Куда нам следует отправиться во время нашего отпуска?

ПОСЛЕ ПЕРЕПИСЫВАНИЯ:

Выступите в роли туристического гида. Опишите отличные места для посещения в Нью-Йорке, Манхэттене с 3-летним ребенком.

Старайтесь использовать глаголы, которые описывают действие. Вот набор примеров:

Действуй, Анализируй, Категоризируй, Классифицируй, Противопоставь, Сравни, Создай, Опиши, Определи, Оцени, Извлеки, Найди, Сгенерируй, Идентифицируй, Перечисли, Измерь, Организуй, Разбери, Выбери, Предскажи, Предоставь, Ранжируй, Рекомендуй, Верни, Извлеки, Перепиши, Выбери, Покажи, Отсортируй, Суммируй, Переведи, Напиши.

Будьте конкретны в отношении вывода

Будьте конкретны в отношении желаемого вывода. Краткая инструкция может недостаточно руководить LLM или быть слишком общей. Предоставление конкретных деталей в промпте (через системный или контекстный промптинг) может помочь модели сосредоточиться на том, что актуально, улучшая общую точность.

Примеры:

ДЕЛАЙТЕ:

Сгенерируйте 3-параграфный блог-пост о 5 лучших игровых консолях. Блог-пост должен быть информативным и увлекательным, и он должен быть написан в разговорном стиле.

НЕ ДЕЛАЙТЕ:

Сгенерируйте блог-пост об игровых консолях.

Используйте инструкции вместо ограничений

Инструкции и ограничения используются в промптинге для руководства выводом LLM.

- **Инструкция** предоставляет явные указания по желаемому формату, стилю или содержанию ответа. Она направляет модель относительно того, что модель должна делать или производить.
- **Ограничение** — это набор ограничений или границ для ответа. Оно ограничивает то, что модель не должна делать или чего следует избегать.

Растущие исследования предполагают, что фокусирование на положительных инструкциях в промптинге может быть более эффективным, чем полагаться в значительной степени на ограничения. Этот подход соответствует тому, как люди предпочитают положительные инструкции спискам того, что не следует делать.

Инструкции непосредственно сообщают о желаемом результате, тогда как ограничения могут оставить модель в догадках о том, что разрешено. Это дает гибкость и поощряет творчество в рамках определенных границ, в то время как ограничения могут ограничивать потенциал модели. Также список ограничений может противоречить друг другу.

Ограничения все еще ценны, но в определенных ситуациях. Для предотвращения генерации моделью вредного или предвзятого контента или когда необходим строгий формат вывода или стиль.

Если возможно, используйте положительные инструкции: вместо того, чтобы говорить модели, что не делать, скажите ей, что делать вместо этого. Это может избежать путаницы и улучшить точность вывода.

ДЕЛАЙТЕ:

Сгенерируйте 1-параграфный блог-пост о 5 лучших игровых консолях. Обсуждайте только консоль, компанию, которая ее создала, год выпуска и общие продажи.

НЕ ДЕЛАЙТЕ:

Сгенерируйте 1-параграфный блог-пост о 5 лучших игровых консолях. Не перечисляйте названия видеоигр.

В качестве лучшей практики начните с приоритезации инструкций, четко указывая, что вы хотите, чтобы модель делала, и используйте ограничения только при необходимости для безопасности, ясности или конкретных требований. Экспериментируйте и итерируйте, чтобы проверить различные комбинации инструкций и ограничений для нахождения того, что лучше всего работает для ваших конкретных задач, и документируйте их.

Контролируйте максимальную длину токенов

Чтобы контролировать длину сгенерированного ответа LLM, вы можете либо установить ограничение максимального количества токенов в конфигурации, либо явно запросить определенную длину в вашем промпте. Например:

"Объясните квантовую физику в сообщении длиной с твит"

Используйте переменные в промптах

Для повторного использования промптов и придания им более динамичного характера используйте переменные в промпте, которые могут быть изменены для различных входных данных. Например, как показано в Таблице 20, промпт, который дает факты о городе. Вместо жесткого кодирования названия города в промпте используйте переменную. Переменные могут сэкономить ваше время и усилия, позволяя избежать повторения. Если вам нужно использовать один и тот же фрагмент информации в нескольких промптах, вы можете сохранить его в переменной и затем ссылаться на эту переменную в каждом промпте. Я очень подробно расскажу о теме семантики промтов в уроке по написанию системных инструкций, но пока что: это имеет смысл при интеграции промптов в ваши собственные приложения, ботов, ассистов, цепочки автоматизаций – так нейронка понимает ваши «казнить, нельзя помиловать». Помните про первые уроки про нейросети в разделе введение (история про голодных рыбов).

Prompt	`VARIABLES`
	`{city} = "Amsterdam"`
	
	`PROMPT`
	`You are a travel guide. Tell me a fact about the city: {city}`

Output	Amsterdam is a beautiful city full of canals, bridges, and narrow streets. It's a great place to visit for its rich history, culture, and nightlife.

Таблица 20. Использование переменных в промптах

Экспериментируйте с форматами ввода и стилями письма

Различные модели, конфигурации моделей, форматы промптов, выбор слов и настройки могут давать разные результаты. Поэтому важно экспериментировать с атрибутами промпта, такими как стиль, выбор слов и тип промпта (zero shot, few shot, системный промпт).

Например, промпт с целью генерации текста о революционной игровой консоли Sega Dreamcast может быть сформулирован как **вопрос**, **утверждение** или **инструкция**, что приведет к разным выводам:

- **Вопрос:** Что такое Sega Dreamcast и почему она была такой революционной консолью?
- **Утверждение:** Sega Dreamcast была игровой консолью шестого поколения, выпущенной Sega в 1999 году. Она...
- **Инструкция:** Напишите один абзац, который описывает консоль Sega Dreamcast и объясняет, почему она была такой революционной.

Адаптируйтесь к обновлениям модели

Важно следить за изменениями архитектуры модели, добавленными данными и возможностями. Попробуйте более новые версии модели и настройте свои промпты, чтобы лучше использовать новые функции модели. Я постоянно тестирую все системные инструкции, промпты, настройки и ежедневно, нет, ежечасно обновляю их с командой разработчиков. Важно всегда понимать – это крайне динамичная индустрия. И действительно важный фактор – постоянная практика.

Экспериментируйте с форматами вывода

Помимо формата ввода промпта, рассмотрите возможность экспериментирования с форматом вывода. Для нетворческих задач, таких как извлечение, выбор, анализ, упорядочивание, ранжирование или категоризация данных, попробуйте получать результаты в структурированном формате, например, JSON или XML.
Существуют определенные преимущества в получении объектов JSON из промпта, который извлекает данные. В реальном приложении мне не нужно вручную создавать этот формат JSON, я уже могу возвращать данные в упорядоченном виде (очень удобно при работе с объектами даты и времени), но самое главное, запрашивая формат JSON, это заставляет модель создавать структуру и ограничивает галлюцинации.
Подводя итог, преимущества использования JSON для вывода:

- Всегда возвращает результаты в одинаковом стиле
- Фокусируется на данных, которые вы хотите получить
- Меньше шансов на галлюцинации
- Делает структуру отношений понятной
- Вы получаете типы данных
- Вы можете сортировать данные

Работа со схемами

Использование структурированного JSON в качестве вывода - отличное решение, как мы видели несколько раз в этом руководстве. Но как насчет *ввода*? Хотя JSON отлично подходит для структурирования *вывода*, который генерирует LLM, он также может быть невероятно полезен для структурирования *ввода*, который вы предоставляете. Именно здесь на сцену выходят JSON-схемы. JSON-схема определяет ожидаемую структуру и типы данных вашего JSON-ввода. Предоставляя схему, вы даете LLM четкий план данных, которые он должен ожидать, помогая ему сфокусировать свое *внимание* на релевантной информации и снижая риск неправильной интерпретации ввода. Кроме того, схемы могут помочь установить отношения между различными фрагментами данных и даже сделать LLM "осведомленным о времени", включая поля даты или временной метки с определенными форматами.

Вот простой пример:

Допустим, вы хотите использовать LLM для создания описаний продуктов в каталоге электронной коммерции. Вместо того, чтобы просто предоставлять текстовое описание продукта в свободной форме, вы можете использовать JSON-схему для определения атрибутов продукта:

```
{
  "type": "object",
  "properties": {
    "name": { "type": "string", "description": "Product name" },
    "category": { "type": "string", "description": "Product category" },
    "price": { "type": "number", "format": "float", "description": "Product
price" },
    "features": {
      "type": "array",
      "items": { "type": "string" },
      "description": "Key features of the product"
    },
    "release_date": { "type": "string", "format": "date", "description":
"Date the product was released"}
  },
```

Сниппет 5. Определение схемы структурированного вывода

Затем вы можете предоставить фактические данные о продукте в виде объекта JSON, который соответствует этой схеме:

```
{
  "name": "Wireless Headphones",
  "category": "Electronics",
  "price": 99.99,
  "features": ["Noise cancellation", "Bluetooth 5.0", "20-hour battery life"],
  "release_date": "2023-10-27"
}
```

Сниппет 6. Структурированный вывод от LLM

Предварительно обрабатывая свои данные и вместо предоставления полных документов, предоставляя как схему, так и данные, вы даете LLM четкое понимание атрибутов продукта, включая дату его выпуска, что делает гораздо более вероятным генерацию точного и релевантного описания. Этот подход к структурированному вводу, направляющий внимание LLM на соответствующие поля, особенно ценен при работе с большими объемами данных или при интеграции LLM в сложные приложения. Понятно? Скорее всего нет, понятие структуры для ЛЛМ мы будем более подробно разбирать на уроке «написание системных инструкций». Так что если тут вам это не понятно – не расстраивайтесь, мы не объясняли подробно.

Четыре группы методик промптинжиниринга и их применение в n8n.

Сразу оговорюсь, в этих уроках НЕ стоит цель сразу объяснить Вам как запускать и пользоваться n8n. Цель этих уроков – дать понимание как в n8n работают техники промтинжиниринга. Подробные гайды по установке и запуску мы разберем позже.

Когда вы освоили основы и архитектуру промптов, возникает следующий вопрос: **а как решать сложные задачи?**
Здесь на помощь приходят **4 стратегических группы техник**, которые позволяют моделям не просто отвечать, а **рассуждать, уточнять, обучаться и адаптироваться**.

Каждая группа:

- Подходит под определённый тип задач
- Легко интегрируется в узлы n8n
- Повышает качество и надёжность результатов

Какие техники мы разобрали на уроках?

- **Zero-Shot Prompting (запрос без примеров)** — техника, когда модель получает только инструкцию/вопрос **без каких-либо примеров** и должна сразу дать ответ.
- **Few-Shot Prompting (запрос с примерами)** — подача модели нескольких примеров или демонстраций задачи в самом промпте, чтобы она лучше поняла формат и контекст перед решением новой задачи.
- **Chain-of-Thought (CoT) Prompting («цепочка мыслей»)** — метод, при котором модель генерирует последовательность промежуточных шагов рассуждений перед финальным ответом, что улучшает способность решать сложные вопросы.
- **Self-Consistency (самоконсистентность)** — подход, когда модель порождает несколько вариантов рассуждений/ответов для одного вопроса и затем выбирается наиболее частый или согласованный результат для повышения надежности ответа.
- **Generated Knowledge Prompting (генерация знаний перед ответом)** — стратегия, при которой сначала запрашивается вспомогательная информация или факты по теме, а затем эта сгенерированная «база знаний» используется для более точного ответа на основной вопрос.
- **Prompt Chaining (цепочка промптов)** — разбиение сложной задачи на серию связанных шагов/промптов, где результат одного шага передается в следующий, чтобы поэтапно прийти к решению.
- **Tree of Thoughts (дерево мыслей)** — расширение идеи цепочки, когда модель рассматривает несколько вариантов рассуждений (ветвится), оценивает их и выбирает лучшую ветку решения, подобно перебору вариантов в дереве решений.
- **Retrieval Augmented Generation (RAG, генерация с поиском)** — интеграция поиска или внешней базы знаний в процесс ответа: модель сначала извлекает (retrieve) релевантную информацию из источников (например, документов, интернет) и затем генерирует ответ на основе этих актуальных данных.

- **Automatic Reasoning and Tool-use (автоматическое рассуждение с инструментами)** — концепция, при которой модель сама решает, какие промежуточные действия выполнить или какие инструменты (калькулятор, поиск) использовать, чтобы прийти к правильному ответу на запрос.
- **Automatic Prompt Engineer (автоматизированное проектирование промптов)** — метод, при котором сама модель (или связка моделей) генерирует и тестирует варианты промптов, чтобы найти наиболее эффективный запрос для решения поставленной задачи.
- **Active-Prompt (активное промптирование)** — подход, вдохновленный активным обучением: целенаправленный выбор примеров (в том числе с участием человека) для few-shot промпта. Модель пробует ответить на ряд вопросов, выявляет где не уверена, и для этих случаев человек добавляет проработанные примеры в промпт для улучшения результата.
- **Directional Stimulus Prompting (направляющий стимул)** — техника, где модели даются специальные подсказки или «стимулы» (например, ключевые слова, намёки) перед формулировкой ответа, чтобы направить генерацию в нужное русло. Часто реализуется с помощью отдельной небольшой модели, которая генерирует такой стимул для основной модели.
- **Program-Aided Language Model (PAL, модель с помощью программирования)** — метод, когда языковая модель для решения задачи генерирует код или псевдокод (например, на Python), который затем выполняется для получения точного результата. Полезен для математических расчётов и задач, требующих точной логики.
- **ReAct Prompting (Reason + Act, рассуждение и действие)** — схема взаимодействия, где модель чередует рассуждения и действия: сначала «думает» (например, планирует шаг), затем выполняет действие (например, вызывает инструмент или делает запрос), снова оценивает результат и так далее, пока не придёт к решению.
- **Reflexion (рефлексия модели)** — процесс, позволяющий модели анализировать и пересматривать свои ответы. Модель получает обратную связь о том, корректен ли ответ, и при необходимости пытается исправить или уточнить свой предыдущий ответ, «размышляя» над своими ошибками.
- **Multimodal CoT Prompting (мультимодальная цепочка мыслей)** — расширение метода CoT для мульти-модальных моделей: модель может рассуждать шаг за шагом, оперируя не только текстом, но и другими типами данных (изображения, аудио). Например, описывать и анализировать картинку последовательными мыслями, прежде чем ответить на вопрос по ней.
- **Graph Prompts (графовые промпты)** — техника, при которой информация структурируется в виде графа (узлы и связи) и подаётся модели в таком структурированном виде или с использованием специального форматирования. Это помогает модели решать задачи, связанные со связями между сущностями, и улучшает понимание сложных структур данных.

Чтобы все это визуализировать, вы можете открыть манд-карту по (ССЫЛКА), где каждая техника подробно расписана и описана.

Группа 1: Основы и Контекст

---

1. Zero-Shot Prompting

Без примеров. Только чёткое задание и формат ответа.

Пример:

```
Определи: письмо важно или нет? Ответь одним словом: IMPORTANT или NOT
IMPORTANT.
```

n8n:

- `IMAP Email` → `Set` → `OpenAI`
- Логика If: по ключевому слову направлять письмо дальше

---

2. Few-Shot Prompting

Даём модели 2–5 примеров, как себя вести

Пример:

```
Пример 1:
Пользователь: "Кто написал «Войну и мир»?"
Ответ: "Лев Толстой"

Пример 2:
Пользователь: "Кто открыл радиоактивность?"
Ответ: "Антуан Анри Беккерель"

Пользователь: "Кто открыл пенициллин?"
Ответ:
```

n8n:

- В `Function` узле собрать строку из примеров + реальный запрос
- Передать в OpenAI или HTTP узел

---

3. Generated Knowledge Prompting

Сначала генерируем "обзор знаний", потом — ответ

Структура:

- УЗЕЛ 1 (LLM): "Сформируй краткий обзор темы..."
- УЗЕЛ 2 (LLM): "Используя обзор, ответь на вопрос..."

n8n:

- Ветка из 2 LLM-узлов, где первый формирует контекст, второй — финальный ответ

---

4. Directional Stimulus Prompting

Задаём тональность, роль, эмоциональную окраску

Пример:

```
Представь, что ты дружелюбный стартап-ментор. Ответь в этом тоне.
```

n8n:

- Передаётся в `system prompt` или как часть пользовательского запроса
- Можно использовать Switch по "тону" (формальный / креативный / резкий)

Группа 2: Логика и Рассуждение

---

5. Chain-of-Thought Prompting

Попросить модель «думать вслух»

Пример:

```
Реши задачу пошагово: если на столе 3 яблока, а ты съел одно, сколько осталось?
```

n8n:

- LLM узел 1: генерация цепочки рассуждений
- LLM узел 2: вывод финального ответа из цепочки

6. Self-Consistency

Получить несколько версий ответа и выбрать наиболее повторяющийся

Идея: запускать 3 параллельных вызова с разными `temperature`

n8n:

- `SplitInBatches` → 3 OpenAI вызова
- `Merge` узел: выбираем наиболее частый ответ
- Идеально для юридических задач или аналитики

---

7. Active Prompting

Модель сама предлагает, какие уточнения ей нужны для ответа

Пример:

```
Чтобы ответить, уточни: какая отрасль и регион интересует?
```

n8n:

- Первый вызов LLM генерирует уточняющий вопрос
- n8n: либо отправляет его пользователю, либо сам ищет ответ
- Второй вызов LLM — уже с полными данными

---

8. Reflexion (Рефлексия)

Модель сама анализирует собственный ответ на ошибки

n8n:

- УЗЕЛ 1: модель генерирует решение
- УЗЕЛ 2: та же модель или другая проверяет это решение
- УЗЕЛ 3 (опционально): доработка, если найдены ошибки

Группа 3: Многошаговые процессы

---

9. Prompt Chaining

Разбиваем задачу на этапы, где каждый шаг — узел

Пример:

1. Считай текст
2. Выдели темы
3. Составь тезисы
4. Преврати их в пост для соцсетей

n8n:

- Каждый шаг = отдельный узел (или подпроцесс через Subworkflow)

---

10. Tree of Thoughts (ToT)

Не одна линия мышления, а параллельные ветви

n8n:

- Создаём 3 ветви генерации по одной и той же задаче
- Затем Merge → анализируем, какая наиболее логична / полная / точная

---

11. Program-Aided Language Model (PAL)

Модель пишет код → n8n исполняет → результат снова в модель

Пример:

```
Напиши JS-функцию, которая считает медиану массива
```

n8n:

- LLM узел: генерирует код
- Code узел: выполняет код (JS/Python)
- Возврат результата обратно в поток

12. Multimodal CoT / Graph Prompting

Модель работает не только с текстом, но и с JSON/изображениями/графами

n8n:

- HTTP Request → получает структуру данных (например, JSON-граф)
- LLM интерпретирует структуру (например: "определи главные узлы сети")
- Можно визуализировать результат через сторонние сервисы

Группа 4: Автоматизация и Инструменты

---

13. Retrieval Augmented Generation (RAG)

Перед генерацией мы извлекаем контекст из базы

Пример:

1. Пользователь спрашивает: "Какая политика возврата?"
2. n8n достаёт фрагмент из Google Sheets или Supabase
3. Только потом передаёт всё в LLM

n8n:

- Google Sheets → OpenAI (вместе с данными)

---

14. Automatic Reasoning + Tool Use

LLM говорит: "Найди это в интернете" — а n8n делает

n8n:

- LLM: предлагает действие
- n8n: парсит команду, вызывает нужный API
- Результат возвращается обратно в LLM

---

15. ReAct (Reason + Act)

Модель рассуждает, затем вызывает инструмент (через n8n)

Пример:

```
- Reason: нужно найти цену акций Apple
- Act: вызвать API Yahoo Finance
```

n8n:

- LLM → Function (парсинг) → HTTP Request → LLM снова

---

16. Automatic Prompt Engineer

LLM сама пишет и тестирует промпты!

Этапы:

1. Сгенерируй 3 промпта для задачи
2. Протестируй на входных данных
3. Выбери лучший
4. Используй его дальше

n8n:

- LLM 1 → 3 промпта
- LLM 2 → тесты
- Switch → лучший → боевое использование

Посмотрели вы на этот список и возник снова вопрос: Денис, мы ничего не поняли. А теперь]

16. Automatic Prompt Engineer
LLM сама пишет и тестирует промпты!
Этапы:
1. Сгенерируй 3 промпта для задачи
2. Протестируй на входных данных
3. Выбери лучший
4. Используй его дальше
n8n:
- LLM 1 → 3 промпта
- LLM 2 → тесты
- Switch → лучший → боевое использование

Запомни, ответ дай максимально кратко

Вот краткий план Auto Prompt Engineer в n8n:

🧠 Этапы:

1. **LLM 1 (OpenAI Node):**
   ➤ Сгенерировать 3 варианта промпта по задаче
   (`Prompt: "Сгенерируй 3 промпта для задачи: {задача}"`)
2. **Split Out (Function Node):**
   ➤ Разделить 3 промпта в массив
3. **LLM 2 (OpenAI Node, в Loop):**
   ➤ Протестировать каждый промпт с одинаковыми входными данными
   (`Prompt: "{тест_промпт} + input: {входные_данные}"`)
4. **Сравнение (Function Node):**
   ➤ Оценить результаты теста (по ключевым метрикам или похожести с эталоном)
5. **Switch Node:**
   ➤ Выбрать лучший промпт по оценке
6. **Использование:**
   ➤ Применить выбранный промпт в боевом сценарии

Что мы сделали? Задали контекст и сгенерировали ассистенту начальные знания.

Далее ставим задачу:

Как мне применить это в задачах отдела продаж?

Отличный вопрос. Вот **ультра-краткий** способ применить Auto Prompt Engineer для задач отдела продаж с помощью n8n:

🎯 Цель:

Авто-подбор наилучшего LLM-промпта для:

- Обработки входящих лидов
- Создания персонализированных писем
- Обработки возражений
- Генерации скриптов для звонков

⚙️ n8n Структура (пример: генерация скрипта звонка):

1. **LLM 1 (OpenAI):**
   - 🔷 Ввод: "Сгенерируй 3 промпта для задачи: 'создать скрипт звонка на основе лида'"
   - 🔷 Выход: `prompt_1`, `prompt_2`, `prompt_3`
2. **Function → Split:**
   - 🔷 Раздели их в массив промптов
3. **LLM 2 (в Loop):**
   - 🔷 Каждый промпт применяется к одинаковому лиду
   - 🔷 Пример: `prompt_i + "Лид: {имя, компания, интерес}"`
4. **Function (Оценка):**
   - 🔷 Оценка по: персонализации, четкости, call-to-action
5. **Switch:**
   - 🔷 Выбор лучшего скрипта
6. **Output/Email/CRM (Боевое):**
   - 🔷 Скрипт отправляется менеджеру или в CRM

🔥 Примеры задач:

Задача	Что делает LLM
Письмо после звонка	Генерирует 3 варианта → тестирует → выбирает
Ответ на возражение	Сравнивает стиль → выбирает лучший
Продажное письмо	LLM сам учится писать лучше по фидбеку

А теперь по аналогии делаем с каждым пунктом и вашей сферой деятельности. Начиная с 1 и заканчивая 16 включительно.

Гибридные стратегии: Комбинируем методы для повышения надёжности и эффективности

Введение

В реальных задачах **одной техники часто недостаточно**.
Нужно **сочетать несколько приёмов**, чтобы:

- Уточнять задачи динамически (Active + Chain-of-Thought)
- Повышать точность (Few-Shot + Self-Consistency)
- Минимизировать стоимость вызовов
- Автоматизировать обучение моделей на лету

Так рождаются **гибридные стратегии промптинжиниринга** — ключ к сложным, устойчивым и масштабируемым системам.

Стратегия 1: Few-Shot + Chain-of-Thought

Идея:

- Модель получает **примеры ответов (Few-Shot)**
- Затем размышляет пошагово (**Chain-of-Thought**)

Пример задачи:
Объясни, почему текст считается манипулятивным.

Промпт:

```
Пример 1:
Текст: "Вы обязаны срочно перевести деньги!"
Анализ: Язык давления, срочность → манипуляция

Пример 2:
Текст: "Будь умнее остальных — купи сейчас!"
Анализ: Эффект превосходства, социальное сравнение → манипуляция

Текст: "{{ $json["message"] }}"
Анализ:
```

n8n:

- `Set` (формирование примеров)
- `LLM` → отвечает пошагово

- `Code` → выделяет итоговое суждение ("манипулятивный / нет")

Стратегия 2: RAG + Prompt Chaining

Идея:

- Сначала извлекаем информацию (**Retrieval Augmented Generation**)
- Потом поэтапно формируем ответ (**Prompt Chaining**)

Пример задачи:
Создать текст email на основе базы знаний и предыдущего диалога.

n8n:

1. `Google Sheets` → находим фрагмент по теме
2. `LLM (Шаг 1)` — формируем краткий конспект
3. `LLM (Шаг 2)` — превращаем его в текст письма
4. `Email` — отправляем

Визуально:
Данные → □ Обзор → □ Сборка письма → □ Отправка

Стратегия 3: Active Prompt + ReAct

Идея:

- Модель сама предлагает, что уточнить или сделать (**Active Prompt**)
- Затем **рассуждает + действует** по схеме ReAct

Пример задачи:
"Какую CRM посоветуешь моей компании?"

n8n:

1. `LLM (Шаг 1)` — генерирует уточняющий вопрос: "Какой размер команды? SaaS или не SaaS?"
2. `LLM (Шаг 2)` — рассуждает: "Если B2B SaaS на 10 человек → подойдет HubSpot"
3. `HTTP` → проверяет актуальные тарифы
4. `LLM (Шаг 3)` — финальный совет

Стратегия 4: Tree of Thoughts + Self-Consistency

Идея:

- Несколько мыслительных ветвей по одной теме (**ToT**)
- Сравнение и выбор наиболее повторяющегося ответа (**Self-Consistency**)

Пример задачи:
"Какой подход лучше всего подойдёт для продвижения бренда на YouTube?"

n8n:

1. Создать 3 ветки генерации стратегии
2. `Merge` → выбрать стратегию, которая:
   - Повторяется
   - Или набрала больше "веса"
   - Или по субъективной оценке — лучше

Можно добавить голосование через `Telegram` или `Webhook`

Стратегия 5: PAL + Reflexion

Идея:

- Модель генерирует код (**Program-Aided Language Model**)
- Потом проверяет и дорабатывает результат (**Reflexion**)

Пример задачи:
"Напиши функцию, которая находит повторяющиеся слова в тексте."

n8n:

1. `LLM (Шаг 1)` — создаёт JS-функцию
2. `Code (JS)` — выполняет
3. `LLM (Шаг 2)` — анализирует, есть ли ошибка
4. Если есть → `LLM (Шаг 3)` генерирует исправление

Практика: Гибридная система анализа откликов

Задача:
Автоматическая обработка фидбэка от пользователей → классификация → план действия

n8n Workflow:

1. Google Forms → n8n Webhook
2. `LLM (Few-Shot)` → классифицирует: жалоба / предложение / благодарность
3. `LLM (Chain-of-Thought)` → объясняет, почему
4. `IF` → в зависимости от категории:
   - Жалоба → Slack + менеджеру
   - Предложение → `LLM (ReAct)` предлагает шаг
   - Благодарность → автоответ с благодарностью

Использовано:

- Few-Shot
- CoT
- ReAct
- Мультиветвление (IF/Switch)

Но что, если ваш workflow в n8n перестанет быть просто «линейной схемой»?
Что, если он превратится в **автоматического агента**, который:

- уточняет вопросы,
- взаимодействует с другими сервисами,
- рассуждает перед ответом,
- вызывает инструменты,
- и даже исправляет себя сам?

Такой подход уже не теория — его можно **визуализировать и реализовать в n8n**.
В этой главе мы покажем, как шаг за шагом **собирать интеллектуальных агентов** с помощью LLM и узлов n8n.

Кто такие интеллектуальные агенты?

Интеллектуальный агент — это **модель поведения**, в которой LLM:

1. Получает ввод (вопрос, событие, задачу)
2. Анализирует ситуацию
3. Генерирует план или намерение
4. Выполняет действия (или делегирует)
5. Проверяет результат и уточняет

Именно этот цикл можно **моделировать в n8n**.

Базовая архитектура агента

Этап	Реализация в n8n
1. Ввод	`Webhook`, `Telegram Trigger`, `Email`
2. Анализ	`OpenAI`, `Claude`, `Function`
3. План действия	`LLM → Function` → JSON-план
4. Выполнение	API-узлы, `HTTP Request`, `Switch`
5. Проверка результата	`LLM` или `If` + `LLM`
6. Финальный вывод / ответ	`Telegram`, `Email`, `Slack`

Пример 1: Telegram-агент-советник

Задача: бот, который уточняет вопрос, предлагает решение и даёт ссылку на продукт

Workflow:

1. `Telegram Trigger` → получаем сообщение
2. `LLM (Active Prompt)` → уточняющий вопрос: "Какой у вас бюджет?"
3. `Telegram` → задаём уточнение
4. Пользователь отвечает → `LLM (Chain-of-Thought)` анализирует
5. `LLM (ReAct)` → "нужен маркетинговый инструмент → ищу в базе"
6. `HTTP Request` к Notion / Airtable / Supabase
7. `LLM` → Финальное сообщение: краткое объяснение + ссылка

Использовано:
Active Prompt + CoT + ReAct + Tool-Use

Пример 2: Агент-копирайтер

Задача: бот, который генерирует пост для соцсети по краткому описанию + адаптирует под стиль бренда

Workflow:

1. `Webhook` / `Google Form` → получаем описание
2. `LLM (Few-Shot)` → генерирует пост в стиле бренда
3. `LLM (Directional Prompt)` → адаптирует под тон: строгий / весёлый / информативный
4. `If` → проверка длины (лимит на символы)
5. `HTTP` → публикация через API (например, Buffer)

Логика принятия решений: как «думать» на лету

Можно дать LLM схему принятия решений. Пример:

```
Если вопрос касается:
- Юридических документов → проконсультировать, не давая точного совета
- Инструментов → выбрать по бюджету и цели
- Общих знаний → просто объяснить
```

В n8n:

- LLM генерирует "decision"
- Switch по типу задачи → подветвь
- Каждая ветвь ведёт к своему API или шаблону ответа

Самообучающийся агент

Агент, который сам выбирает лучший способ решить задачу

Workflow:

1. `LLM 1` — генерирует 3 варианта промпта
2. `LLM 2` — тестирует их на входных данных
3. `Switch` → лучший → отправка в боевую зону
4. `Code` → сохраняет этот промпт как лучший для будущих вызовов

Использовано:
Automatic Prompt Engineer + Reflexion + Memory

Пример 3: Финансовый ассистент с агентной логикой

Задача: собрать данные о транзакциях → распознать расходы → составить рекомендации

Workflow:

1. `Webhook` или `Email` с выпиской
2. `Code` → преобразование в JSON
3. `LLM` → классификация транзакций (еда, подписки, услуги)
4. `LLM` → совет: где можно сэкономить
5. `Telegram` → отправка отчёта

Кейс 1: Поддержка клиентов — фильтрация и автоответы

Задача:

Автоматически фильтровать обращения клиентов по категории и срочности, формировать ответ.

Workflow:

1. `IMAP Email` → получение письма
2. `LLM (Few-Shot)` → анализ категории (вопрос / жалоба / благодарность)
3. `LLM (Zero-Shot)` → определение важности (URGENT / OK)
4. `Switch` → логика маршрутизации:
   - Жалоба + URGENT → Slack
   - Благодарность → автоответ
   - Вопрос → ответ из базы (RAG)
5. `Gmail / Telegram` → ответ пользователю

Техники: Few-Shot + Directional Style + Conditional Flow + ReAct (база знаний)

Результат:
До 70% писем обрабатываются без участия человека.

Кейс 2: Генерация постов и контента

Задача:

Создавать соцмедийные посты по описанию продукта, адаптировать под стиль и публиковать.

Workflow:

1. `Webhook` / `Google Sheets` → вход: «Продукт: CRM-система для стартапов»
2. `LLM (Few-Shot)` → генерирует пост
3. `LLM (Directional)` → адаптирует под нужный стиль (весёлый / экспертный / лаконичный)
4. `LLM (Reflexion)` → проверка: соответствует ли длина, CTA, стиль
5. `Buffer API` → отложенная публикация

Техники: Prompt Chaining + Reflexion + Style Control

Результат:
Генерация 10–15 постов в день без участия копирайтера

Кейс 3: Финансовая аналитика по выпискам

Задача:

Обработать банковские выписки → классифицировать расходы → предложить экономию.

Workflow:

1. `Email` с CSV-выпиской → `Code` → JSON-массив транзакций
2. `LLM` → классификация: еда / подписки / личное / реклама
3. `Group By` → подсчёт по категориям
4. `LLM` → советы по оптимизации расходов
5. `Google Slides` → формирование отчёта

Техники: PAL (code generation) + CoT + Summary Prompting

Результат:
Автоматический финотчёт на 2 страницы за 3 секунды

Кейс 4: Ассистент по юридическим договорам

Задача:

Проанализировать договор → определить риски → предложить формулировки

Workflow:

1. `Upload` PDF → `PDF Parse` → текст
2. `LLM (Chain-of-Thought)` → пошаговый анализ: есть ли риски, где двусмысленность
3. `LLM (Few-Shot)` → предложить корректные формулировки
4. `Code` → компоновка отчёта
5. `Email` / `Notion` → выгрузка результата

Техники: Generated Knowledge → CoT → Correction Prompts

Результат:
До 80% рутинной правовой аналитики автоматизировано

Кейс 5: Рекрутер-бот

Задача:

Автоматизировать ответы на входящие резюме, присылать кандидату фидбэк и вопросы.

Workflow:

1. `Email` / `Telegram` → получение резюме (PDF/DOCX)
2. `LLM` → извлечение опыта, навыков, проектов
3. `LLM` → сравнение с позицией (в базе)
4. `Switch`:
   - Недостаточно опыта → вежливый отказ
   - Частичное совпадение → фоллоу-ап с вопросами
   - Полное совпадение → отправка тестового задания

Техники: RAG + Reflexion + Directional Prompt + Self-Consistency (внутренние оценки)

Результат:
Рекрутер экономит 3–4 часа в день

Отточив эти знания на практике, вы теперь станете не просто «пользователем LLM, а **архитектором интеллектуальных сценариев**, способным:

- Формировать умные промпты
- Строить сложные workflows
- Автоматизировать реальные бизнес-процессы
- Увеличивать эффективность команд
- Делать ИИ доступным каждому — через визуальный подход

С помощью n8n вы превратите взаимодействие с моделью из «чёрного ящика» в **понятную, контролируемую, мощную автоматизацию**.

ИИ — это не будущее. Это инструмент, который можно проектировать сегодня.Эволюционируй или умри.